LAPORAN HASIL TUGAS
PROBABILITAS DAN DISTRIBUSI PROBABILITAS PYTHON
MATA KULIAH PROBABILITAS DAN STATISTIKA



Dosen Pengampu :

Mona Elviyenti, S.Si, M.Si

Disusun Oleh :

Nama : Nazwa Amalya Shaliha

Kelas : 3 Teknik Informatika - F

NIM : 2455301231

Program Studi Teknik Informatika

POLITEKNIK CALTEX RIAU

TA 2026 / 2027

BAB I

DAFTAR LIBRARY PYTHON

Berikut library yang dieksplorasi beserta fungsi utama, syntax pemanggilan, dan contoh penggunaannya. Seluruh contoh telah dijalankan pada notebook.

Library

Library	Fungsi / kegunaan	Syntax pemanggilan	Contoh dalam tugas
math	Bawaan Python. Faktorial, permutasi, kombinasi, eksponen, logaritma.	import math math.factorial(5) # 120 math.perm(10, 3) # 720 math.comb(12, 4) # 495 math.exp(-2)	Menghitung C(52,5) = 2.598.960 pada soal B4.
itertools	Bawaan Python. Membentuk ruang sampel: product, permutations, combinations.	import itertools S = list(itertools.product(range(1,7), repeat=2)) list(itertools.permutations(range(5), 3)) list(itertools.combinations(range(12), 4))	Ruang sampel dua dadu (36 pasangan) pada soal A1.
fractions	Bawaan Python. Peluang sebagai pecahan eksak (tanpa galat desimal).	from fractions import Fraction Fraction(6, 36) # Fraction(1, 6)	P(jumlah=7) = 1/6 pada A1.
collections	Bawaan Python. Counter untuk menghitung frekuensi kemunculan.	from collections import Counter Counter(['merah','biru','merah'])	Menghitung frekuensi kemunculan tiap kejadian pada soal A0B.
random	Bawaan Python. Pembangkit bilangan acak sederhana.	import random random.seed(42) random.randint(1, 6) random.choice(['A','G'])	Seed dipasang agar hasil simulasi dapat diulang.
statistics	Bawaan Python. Mean, median, variance untuk data kecil.	import statistics as st st.mean([1,2,3,4]) st.variance([1,2,3,4])	Alternatif ringan selain numpy untuk statistik dasar.
numpy	Array numerik, simulasi acak skala besar, mean/var/std, cumsum.	import numpy as np rng = np.random.default_rng(42) rng.integers(1, 7, 100000) rng.binomial(10, 0.3, 100000)	Simulasi 1.000.000 titik untuk menaksir π (G1).

		np.mean(x), np.var(x), np.cumsum(x)	
scipy.stats	Objek distribusi: pmf/pdf, cdf, sf, ppf, mean, var, std, stats, rvs.	from scipy import stats stats.binom.pmf(2, 10, 0.1) stats.poisson.cdf(2, 4) stats.geom.sf(10, 0.2) stats.hypergeom.pmf(1, 50, 8, 5) stats.uniform(loc=0, scale=10).cdf(3) stats.norm.ppf(0.9, 70, 10) stats.expon(scale=2).sf(3) stats.binom.rvs(10, 0.3, size=1000)	Dipakai pada hampir seluruh soal di Bab III (distribusi probabilitas).
pandas	Tabel data, tabel kontingensi, ringkasan hasil simulasi.	import pandas as pd pd.DataFrame(...) pd.crosstab(a, b) pd.Series(x).value_counts(normalize=True)	Tabel bug per modul pada soal D3.
matplotlib	Grafik: bar, histogram, line, scatter, fill_between.	import matplotlib.pyplot as plt plt.bar(k, pmf) plt.hist(data, density=True) plt.plot(x, pdf) plt.show()	Grafik pmf/pdf dan histogram simulasi vs teori.

Fungsi utama pada scipy.stats

Fungsi	Arti	Dipakai untuk pertanyaan
pmf(k, ...)	Peluang tepat k (diskrit)	P(X = k)
pdf(x, ...)	Kepadatan (kontinu), bukan peluang	Bentuk kurva
cdf(x, ...)	`P(X ≤ x)`	`'paling banyak'`, `'kurang dari'`, `P(a<X<b) = cdf(b) − cdf(a)`
sf(x, ...)	`P(X > x) = 1 − cdf`	`'lebih dari'`, `'minimal'` `(P(X≥k) = sf(k−1) pada diskrit)`
ppf(q, ...)	Kuantil / invers cdf	'nilai batas persentil ke-q'
mean(), var(), std()	Nilai harapan, varians, simpangan baku	E[X], Var(X)
rvs(size=n)	Membangkitkan sampel acak	Simulasi dan histogram

BAB II

PERMASALAHAN PROBABILITAS

Tujuh topik mengikuti modul 04 Probabilitas (total 34 soal). Soal berkode berakhiran 0 (A0, A0B, B0, B0B, D0, F0, F0B, G0) diambil dari contoh pada modul. Notebook: Probabilitas_Python.ipynb.

Topik A — Probabilitas Dasar (Klasik)

Soal A0 : Tabel Frekuensi Relatif Keterlambatan Bus

Soal: Modul mencatat keterlambatan kedatangan bus pada 30 jadwal: keterlambatan 1 sampai 10 menit dengan frekuensi 2, 3, 8, 4, 5, 3, 2, 0, 1, 2. Susun tabel frekuensi relatif (probabilitas empiris), lalu hitung prob(keterlambatan = 3 menit), prob(keterlambatan ≤ 3 menit), dan prob(keterlambatan ≥ 6 menit).

Hitungan manual:

Frekuensi relatif = frekuensi / 30. Jumlah frekuensi = 2+3+8+4+5+3+2+0+1+2 = 30.

prob(3 menit) = 8/30 = 0,2667

prob(≤ 3) = (2+3+8)/30 = 13/30 = 0,4333

prob(≥ 6) = (3+2+0+1+2)/30 = 8/30 = 0,2667

Probabilitas ini adalah perkiraan berdasarkan observasi, bukan himpunan semua kemungkinan.

Source code:

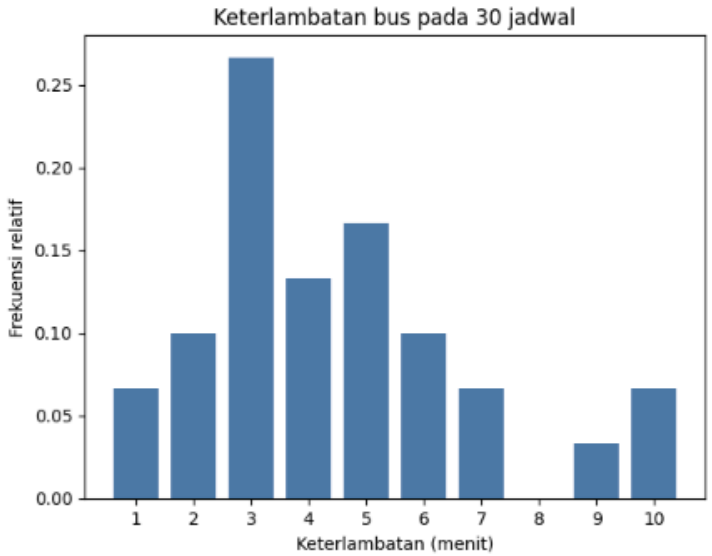
```python
frek = pd.DataFrame({"Keterlambatan (menit)": range(1, 11),
                     "Frekuensi": [2, 3, 8, 4, 5, 3, 2, 0, 1, 2]}).set_index("Keterlambatan (menit)")
n_jadwal = frek["Frekuensi"].sum()
frek["Frek. relatif"] = (frek["Frekuensi"] / n_jadwal).round(2)
frek["Persentase"] = (100 * frek["Frekuensi"] / n_jadwal).round(0).astype(int).astype(str) + "%"
print(frek)
a0_total = (frek["Frekuensi"] / n_jadwal).sum()
a0_rel3 = frek.loc[3, "Frekuensi"] / n_jadwal
a0_le3 = frek.loc[1:3, "Frekuensi"].sum() / n_jadwal
a0_ge6 = frek.loc[6:10, "Frekuensi"].sum() / n_jadwal
print(f"Jumlah frekuensi = {n_jadwal} | jumlah frek. relatif = {a0_total:.4f}")
print(f"prob(3 menit) = {a0_rel3:.4f} | prob(<=3) = {a0_le3:.4f} | prob(>=6) = {a0_ge6:.4f}")

plt.bar(frek.index, frek["Frekuensi"] / n_jadwal, color="#4C78A8")
plt.xlabel("Keterlambatan (menit)"); plt.ylabel("Frekuensi relatif"); plt.xticks(range(1, 11))
plt.title("Keterlambatan bus pada 30 jadwal")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (jumlah frek. relatif):", cocok(1, a0_total, 1e-09))
print("Cek vs manual (prob(3 menit)):", cocok(8/30, a0_rel3, 1e-09))
print("Cek vs manual (prob(<=3)):", cocok(13/30, a0_le3, 1e-09))
print("Cek vs manual (prob(>=6)):", cocok(8/30, a0_ge6, 1e-09))
```

Output :

```
                     Frekuensi  Frek. relatif Persentase
Keterlambatan (menit)
1                            2           0.07         7%
2                            3           0.10        10%
3                            8           0.27        27%
4                            4           0.13        13%
5                            5           0.17        17%
6                            3           0.10        10%
7                            2           0.07         7%
8                            0           0.00         0%
9                            1           0.03         3%
10                           2           0.07         7%
Jumlah frekuensi = 30 | jumlah frek. relatif = 1.0000
prob(3 menit) = 0.2667 | prob(<=3) = 0.4333 | prob(>=6) = 0.2667
```




```
Cek vs manual (jumlah frek. relatif): SAMA
Cek vs manual (prob(3 menit)): SAMA
Cek vs manual (prob(<=3)): SAMA
Cek vs manual (prob(>=6)): SAMA
```

Analisis / Kesimpulan :

Dari tabel output, keterlambatan 3 menit paling sering terjadi, yaitu 8 dari 30 jadwal atau 0,2667 (27%). Jumlah seluruh frekuensi relatif tepat 1,0000, jadi semua kemungkinan keterlambatan sudah tercatat dan tidak ada yang tertinggal.

Peluang bus terlambat paling lama 3 menit adalah 0,4333 (43,3%), sedangkan peluang terlambat 6 menit atau lebih adalah 0,2667 (26,7%). Angka ini hanya perkiraan dari 30 jadwal yang dicatat, jadi kalau datanya diganti nilainya bisa sedikit bergeser. Hasil Python sama dengan hitungan manual (semua cek bernilai SAMA).

Soal A0B : Sample space dan event pada tiga stasiun

Soal: Sebuah DAS memiliki 3 stasiun. Tiap stasiun perlu (y) atau tidak perlu (n) penggantian alat. Anggap kedelapan hasil berbobot sama. (a) Tuliskan sample space S1 dan S2 (jumlah stasiun yang perlu diganti). (b) Hitung prob event A (paling sedikit 2 stasiun perlu diganti), B (tidak ada yang perlu diganti), dan C (tepat 2 stasiun perlu diganti).

Hitungan manual:

S1 memuat $2^3 = 8$ titik sampel, tiap titik berbobot 1/8. S2 = {0, 1, 2, 3} dengan bobot 1/8, 3/8, 3/8, 1/8.

A = {yyy, yyn, yny, nyy} → prob(A) = 4/8 = 0,5

B = {nnn} → prob(B) = 1/8 = 0,125

C = {yyn, yny, nyy} → prob(C) = 3/8 = 0,375

Dengan S2: prob(A) = prob(2) + prob(3) = 3/8 + 1/8 = 0,5 (sama, tetapi informasi stasiun mana yang perlu diganti hilang).

Source code:

```python
S1 = list(itertools.product("yn", repeat=3))          # sample space alternatif 1 (8 titik sampel)
print("S1 =", ["".join(s) for s in S1])
A = [s for s in S1 if s.count("y") >= 2]
B = [s for s in S1 if s.count("y") == 0]
C = [s for s in S1 if s.count("y") == 2]
a0b_A, a0b_B, a0b_C = Fraction(len(A), len(S1)), Fraction(len(B), len(S1)), Fraction(len(C), len(S1))
print(f"prob(A) = {a0b_A} | prob(B) = {a0b_B} | prob(C) = {a0b_C}")

S2 = Counter(s.count("y") for s in S1)                # sample space alternatif 2 = {0,1,2,3}
bobot_S2 = {k: Fraction(v, len(S1)) for k, v in sorted(S2.items())}
print("S2 dan bobot:", {k: str(v) for k, v in bobot_S2.items()}, "| jumlah bobot =", sum(bobot_S2.values()))
a0b_A2 = bobot_S2[2] + bobot_S2[3]
print("prob(A) lewat S2 =", a0b_A2)

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (prob(A)):", cocok(4/8, a0b_A, 1e-09))
print("Cek vs manual (prob(B)):", cocok(1/8, a0b_B, 1e-09))
print("Cek vs manual (prob(C)):", cocok(3/8, a0b_C, 1e-09))
print("Cek vs manual (prob(A) lewat S2):", cocok(3/8 + 1/8, a0b_A2, 1e-09))
```

Output :

```
S1 = ['yyy', 'yyn', 'yny', 'ynn', 'nyy', 'nyn', 'nny', 'nnn']
prob(A) = 1/2 | prob(B) = 1/8 | prob(C) = 3/8
S2 dan bobot: {0: '1/8', 1: '3/8', 2: '3/8', 3: '1/8'} | jumlah bobot = 1
prob(A) lewat S2 = 1/2
Cek vs manual (prob(A)): SAMA
Cek vs manual (prob(B)): SAMA
Cek vs manual (prob(C)): SAMA
Cek vs manual (prob(A) lewat S2): SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan, peluang minimal 2 stasiun yang memerlukan penggantian alat adalah sebesar 50%. Sementara itu, peluang tidak ada stasiun yang perlu diganti bernilai lebih kecil yaitu 12,5%, dan peluang tepat 2 stasiun yang diganti adalah 37,5%.

Penggunaan ruang sampel alternatif (S2) terbukti lebih ringkas dan tetap menghasilkan nilai peluang yang sama (50%) untuk kejadian A, meskipun kelemahannya adalah hilangnya detail informasi mengenai stasiun mana saja yang secara spesifik diganti. Total keseluruhan probabilitas pada setiap ruang sampel akan selalu bernilai pas 1. Sebagai catatan, perhitungan ini menggunakan asumsi bahwa setiap stasiun memiliki peluang 50% untuk diganti secara independen, yang sengaja dibuat untuk menyederhanakan contoh kasus matematis.

Soal A1 : Jumlah dua dadu = 7

Soal: Dua dadu bersisi 6 dilempar bersamaan. Berapa peluang jumlah mata kedua dadu sama dengan 7?

Hitungan manual:

$n(S) = 6 \times 6 = 36$. Pasangan berjumlah 7: $(1,6),(2,5),(3,4),(4,3),(5,2),(6,1) \rightarrow n(A) = 6$.

P(A) = 6/36 = 1/6 ≈ 0,1667

Source code:

```
S = list(itertools.product(range(1, 7), repeat=2))     # ruang sampel 36 pasangan
A = [s for s in S if sum(s) == 7]
a1_p = Fraction(len(A), len(S))
print("n(S) =", len(S), "| n(A) =", len(A))
print("P(jumlah=7) =", a1_p, "=", round(float(a1_p), 4))
print("Manual vs Python:", cocok(1/6, a1_p))
```

Output :

```
n(S) = 36 | n(A) = 6
P(jumlah=7) = 1/6 = 0.1667
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Peluang munculnya jumlah mata dadu 7 adalah sebesar 1/6 atau setara dengan 16,7%. Angka 7 menjadi hasil yang paling mungkin atau paling sering muncul ketika dua buah dadu dilempar bersamaan, hal ini dikarenakan angka 7 memiliki variasi kombinasi terbanyak yaitu 6 kombinasi dari total 36 kemungkinan hasil lemparan (ruang sampel). Hasil perhitungan peluang menggunakan program Python ini juga terbukti akurat dan sama persis dengan hasil perhitungan yang dilakukan secara manual.

Soal A2 : Tiga lemparan koin

Soal: Sebuah koin adil dilempar 3 kali. Berapa peluang muncul tepat 2 kali Angka (A)?

Hitungan manual:

n(S) = 2^3 = 8. Hasil dengan tepat dua A: AAG, AGA, GAA → n(E) = 3.

P(E) = 3/8 = 0,375

Source code:

```
S = list(itertools.product("AG", repeat=3))
E = [s for s in S if s.count("A") == 2]
a2_p = Fraction(len(E), len(S))
print("Ruang sampel:", ["".join(s) for s in S])
print("Kejadian    :", ["".join(s) for s in E])
print("P(tepat 2 Angka) =", a2_p, "=", float(a2_p))
print("Manual vs Python:", cocok(3/8, a2_p))
```

Output :

```
Ruang sampel: ['AAA', 'AAG', 'AGA', 'AGG', 'GAA', 'GAG', 'GGA', 'GGG']
Kejadian    : ['AAG', 'AGA', 'GAA']
P(tepat 2 Angka) = 3/8 = 0.375
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Peluang munculnya tepat dua sisi Angka dari tiga kali lemparan koin adalah sebesar 3/8 atau 37,5%. Berdasarkan perhitungan, dari total 8 kemungkinan hasil lemparan yang ada, hanya terdapat 3 kombinasi yang memenuhi syarat tersebut. Hal ini menunjukkan bahwa kejadian munculnya tepat dua Angka cukup sering terjadi (dengan persentase 37,5%), namun tidak mendominasi karena bukan merupakan kemungkinan hasil yang paling sering muncul.

Soal A3 : Pengambilan satu kartu remi

Soal: Satu kartu diambil acak dari 52 kartu remi. Berapa peluang terambil kartu As, dan berapa peluang terambil kartu Hati?

Hitungan manual:

P(As) = 4/52 = 1/13 ≈ 0,0769

P(Hati) = 13/52 = 1/4 = 0,25

Source code:

```
jenis = ["Hati", "Wajik", "Keriting", "Sekop"]
nilai = ["A"] + [str(i) for i in range(2, 11)] + ["J", "Q", "K"]
kartu = list(itertools.product(nilai, jenis))
a3_as = Fraction(sum(1 for k in kartu if k[0] == "A"), len(kartu))
a3_hati = Fraction(sum(1 for k in kartu if k[1] == "Hati"), len(kartu))
print("Jumlah kartu:", len(kartu))
print("P(As)   =", a3_as, "=", round(float(a3_as), 4))
print("P(Hati) =", a3_hati, "=", float(a3_hati))
print("Manual vs Python:", cocok(1/13, a3_as), "dan", cocok(0.25, a3_hati))
```

Output :

```
Jumlah kartu: 52
P(As)   = 1/13 = 0.0769
P(Hati) = 1/4 = 0.25
Manual vs Python: SAMA dan SAMA
```

Analisis / Kesimpulan :

Peluang terambilnya kartu As dari satu set kartu remi adalah sebesar 1/13 atau sekitar 7,7%. Di sisi lain, peluang terambilnya kartu Hati jauh lebih besar, yakni 1/4 atau setara dengan 25%. Tingginya peluang terambilnya kartu Hati ini sangat wajar mengingat jumlah kartu Hati dalam satu dek jauh lebih banyak (13 kartu) dibandingkan dengan jumlah kartu As (hanya 4 kartu). Selain itu, hasil perhitungan yang dieksekusi melalui program Python terbukti akurat dan sama persis dengan hasil perhitungan manual.

Soal A4 : Menebak PIN 4 digit

Soal: Sebuah PIN terdiri dari 4 digit (0–9, boleh berulang). (a) Berapa peluang menebak PIN yang benar pada satu kali tebakan? (b) Berapa peluang sebuah PIN memiliki semua digit berbeda?

Hitungan manual:

(a) Jumlah PIN = 10^4 = 10.000 → P = 1/10.000 = 0,0001

(b) Digit berbeda: 10 × 9 × 8 × 7 = 5.040 → P = 5.040/10.000 = 0,504

Source code:

```
a4_total = 10**4
a4_a = Fraction(1, a4_total)
semua = itertools.product(range(10), repeat=4)
a4_berbeda = sum(1 for p in semua if len(set(p)) == 4)
a4_b = Fraction(a4_berbeda, a4_total)
a4_rep = 1 - a4_b
print("Total PIN        :", a4_total)
print("(a) P(tebakan benar) =", float(a4_a))
print("(b) PIN digit berbeda:", a4_berbeda, "-> P =", float(a4_b))
print("Manual vs Python:", cocok(0.0001, a4_a), "dan", cocok(0.504, a4_b))
```

Output :

```
Total PIN        : 10000
(a) P(tebakan benar) = 0.0001
(b) PIN digit berbeda: 5040 -> P = 0.504
Manual vs Python: SAMA dan SAMA
```

Analisis / Kesimpulan :

Pada kasus pertama (a), peluang keberhasilan seseorang dalam menebak PIN 4 digit secara acak sangatlah kecil, yaitu hanya sebesar 0,0001 (atau 1 berbanding 10.000). Angka probabilitas yang rendah ini menunjukkan bahwa sistem keamanan PIN 4 digit tergolong lemah dan rentan diretas jika menghadapi serangan *brute force* yang tidak membatasi jumlah percobaan tebakan.

Pada kasus kedua (b), dari total 10.000 kemungkinan kombinasi PIN yang ada, sebanyak 5.040 PIN terbentuk dari susunan 4 digit angka yang semuanya berbeda (tidak berulang). Hal ini berarti terdapat 50,4% peluang sebuah PIN tidak memiliki angka kembar. Sebaliknya, sisa persentasenya yaitu 49,6% dari total kemungkinan, justru merupakan kombinasi PIN yang memiliki setidaknya satu digit angka yang berulang (kembar).

Topik B — Permutasi dan Kombinasi

Soal B0 : Empat cara pengambilan sampel: 2 dari 4 stasiun (contoh modul)

Soal: Dipilih 2 stasiun dari 4 stasiun (A, B, C, D) untuk diberi dana. Hitung banyaknya kemungkinan untuk empat kasus pada modul: (1) urutan penting dengan pengembalian, (2) urutan penting tanpa pengembalian, (3) urutan tidak penting tanpa pengembalian, (4) urutan tidak penting dengan pengembalian.

Hitungan manual:

(1) $n^r = 4^2 = 16$

(2) Permutasi $n!/(n-r)! = 4!/2! = 12$

(3) Kombinasi $n!/((n-r)!\ r!) = 4!/(2!\ 2!) = 6$

(4) Kombinasi dengan pengembalian $(n+r-1)!/((n-1)!\ r!) = 5!/(3!\ 2!) = 10$

Source code:

```
In [11]:  st = ["A", "B", "C", "D"]
          r_st = 2
          kasus = {
              "1. Urutan penting, dengan pengembalian":      list(itertools.product(st, repeat=r_st)),
              "2. Urutan penting, tanpa pengembalian":       list(itertools.permutations(st, r_st)),
              "3. Urutan tak penting, tanpa pengembalian":   list(itertools.combinations(st, r_st)),
              "4. Urutan tak penting, dengan pengembalian":  list(itertools.combinations_with_replacement(st,
          r_st)),
          }
          rumus = [4**2, math.perm(4, 2), math.comb(4, 2), math.comb(4 + 2 - 1, 2)]
          for (nama, daftar), rms in zip(kasus.items(), rumus):
              print(f"{nama}: {len(daftar)} cara (rumus = {rms})")
              print("    ", ["".join(t) for t in daftar])
          b0_1, b0_2, b0_3, b0_4 = (len(d) for d in kasus.values())
          print("Cacah = rumus untuk keempat kasus?", [len(d) for d in kasus.values()] == rumus)
          print("Padanan Excel: PERMUT(4,2) =", math.perm(4, 2), "| COMBIN(4,2) =", math.comb(4, 2), "| FACT(4)
          =", math.factorial(4))

          # Pembanding dengan angka pada hitungan manual
          print("Cek vs manual (kasus 1, toleransi 0.5):", cocok(16, b0_1, 0.5))
          print("Cek vs manual (kasus 2, toleransi 0.5):", cocok(12, b0_2, 0.5))
          print("Cek vs manual (kasus 3, toleransi 0.5):", cocok(6, b0_3, 0.5))
          print("Cek vs manual (kasus 4, toleransi 0.5):", cocok(10, b0_4, 0.5))
```

Output :

```
Out [11]:  1. Urutan penting, dengan pengembalian: 16 cara (rumus = 16)
               ['AA', 'AB', 'AC', 'AD', 'BA', 'BB', 'BC', 'BD', 'CA', 'CB', 'CC', 'CD', 'DA', 'DB', 'DC', 'DD']
           2. Urutan penting, tanpa pengembalian: 12 cara (rumus = 12)
               ['AB', 'AC', 'AD', 'BA', 'BC', 'BD', 'CA', 'CB', 'CD', 'DA', 'DB', 'DC']
           3. Urutan tak penting, tanpa pengembalian: 6 cara (rumus = 6)
               ['AB', 'AC', 'AD', 'BC', 'BD', 'CD']
           4. Urutan tak penting, dengan pengembalian: 10 cara (rumus = 10)
               ['AA', 'AB', 'AC', 'AD', 'BB', 'BC', 'BD', 'CC', 'CD', 'DD']
          Cacah = rumus untuk keempat kasus? True
          Padanan Excel: PERMUT(4,2) = 12 | COMBIN(4,2) = 6 | FACT(4) = 24
          Cek vs manual (kasus 1, toleransi 0.5): SAMA
          Cek vs manual (kasus 2, toleransi 0.5): SAMA
          Cek vs manual (kasus 3, toleransi 0.5): SAMA
          Cek vs manual (kasus 4, toleransi 0.5): SAMA
```

Analisis / Kesimpulan :

Keempat kasus menghasilkan 16, 12, 6, dan 10 kemungkinan. Yang terbanyak adalah urutan penting dengan pengembalian (16), dan yang tersedikit adalah kombinasi tanpa pengembalian (6) karena AB dan BA dianggap sama. Daftar pasangan dari itertools cocok dengan rumus n^r, n!/(n−r)!, C(n,r), dan (n+r−1)!/((n−1)! r!); semua cek bernilai SAMA.

Padanan Excel: math.factorial = FACT, math.perm = PERMUT, dan math.comb = COMBIN (PERMUT(4,2) = 12, COMBIN(4,2) = 6).

Soal B0B : Pendekatan Stirling untuk faktorial

Soal: Modul memberi pendekatan Stirling $n! \approx \sqrt{(2\pi)} \cdot e^{(-n)} \cdot n^{(n+1/2)}$.

Bandingkan dengan $n!$ eksak untuk $n = 5, 10, 20$.

Hitungan manual:

$n = 5$: $\sqrt{(2\pi)} = 2{,}5066$; $e^{-5} = 0{,}006738$; $5^{5{,}5} = 6.987{,}7$ → hasil ≈ 118,02 (eksak 120)

Galat relatif = $(120 - 118{,}02)/120 \approx 1{,}65\%$.

Source code:

```
baris = []
for n in (5, 10, 20):
    eksak = math.factorial(n)
    stirling = math.sqrt(2 * math.pi) * math.exp(-n) * n ** (n + 0.5)
    baris.append((n, eksak, stirling, abs(eksak - stirling) / eksak))
tab = pd.DataFrame(baris, columns=["n", "n! eksak", "Stirling", "galat relatif"])
print(tab.to_string(index=False, formatters={"n! eksak": "{:.4e}".format, "Stirling": "{:.4e}".format, "galat relatif": "{:.2%}".format}))
b0b_n5 = baris[0][2]
b0b_e5, b0b_e10, b0b_e20 = baris[0][3], baris[1][3], baris[2][3]

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (Stirling n=5, toleransi 0.01):", cocok(118.02, b0b_n5, 0.01))
```

Output :

```
 n   n! eksak   Stirling galat relatif
 5 1.2000e+02 1.1802e+02         1.65%
10 3.6288e+06 3.5987e+06         0.83%
20 2.4329e+18 2.4228e+18         0.42%
Cek vs manual (Stirling n=5, toleransi 0.01): SAMA
```

Analisis / Kesimpulan :

Rumus Stirling memberi 118,02 untuk 5!, sedangkan nilai eksaknya 120, sehingga galatnya 1,65%. Galat itu mengecil menjadi 0,83% saat n = 10 dan 0,42% saat n = 20. Jadi makin besar n, hasil Stirling makin dekat ke nilai eksak, dan rumus ini berguna ketika n! terlalu besar untuk dihitung langsung.

Soal B1 : Urutan pengerjaan tugas

Soal: Ada 5 tugas berbeda yang harus dikerjakan satu per satu. Berapa banyak urutan pengerjaan yang mungkin?

Hitungan manual:

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Source code:

```
b1_n = math.factorial(5)
b1_cek = len(list(itertools.permutations(range(5))))
print("5! =", b1_n, "| cacah dengan itertools.permutations =", b1_cek)
print("Manual vs Python:", cocok(120, b1_n))
```

Output :

```
5! = 120 | cacah dengan itertools.permutations = 120
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan di atas, diperoleh sebanyak 120 kemungkinan urutan pengerjaan yang berbeda untuk 5 tugas tersebut. Hasil pencacahan secara langsung menggunakan fungsi itertools.permutations pada program Python terbukti akurat dan menghasilkan angka yang sama persis dengan perhitungan rumus faktorial secara manual, yakni 120. Hal yang menarik untuk diperhatikan adalah jumlah kemungkinan urutan ini akan bertumbuh dengan sangat cepat seiring bertambahnya jumlah tugas; sebagai contoh, jika terdapat 10 tugas, maka variasi urutan pengerjaannya akan melonjak secara drastis hingga mencapai 3.628.800 kemungkinan.

Soal B2 : Memilih pengurus (permutasi)

Soal: Dari 10 mahasiswa dipilih ketua, wakil, dan sekretaris (jabatan berbeda). Berapa banyak susunan pengurus?

Hitungan manual:

$P(10,3) = 10 \times 9 \times 8 = 720$

Source code:

```
b2_n = math.perm(10, 3)
b2_cek = len(list(itertools.permutations(range(10), 3)))
print("P(10,3) =", b2_n, "| cek itertools =", b2_cek)
print("Manual vs Python:", cocok(720, b2_n))
```

Output / grafik:

```
P(10,3) = 720 | cek itertools = 720
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Berdasarkan hasil perhitungan, terdapat 720 kemungkinan susunan pengurus yang dapat dibentuk dari 10 orang mahasiswa. Pada kasus ini, konsep permutasi digunakan karena urutan penempatan sangatlah penting, mengingat adanya tiga posisi jabatan yang secara spesifik berbeda (ketua, wakil, dan sekretaris). Oleh karena itu, jika kelompok orang yang sama terpilih namun menempati posisi jabatan yang saling bertukar (misalnya A sebagai ketua dan B sebagai wakil, dibandingkan dengan B sebagai ketua dan A sebagai wakil), hal tersebut akan tetap dihitung sebagai susunan pengurus yang sepenuhnya berbeda. Hasil eksekusi pada kode Python juga terbukti akurat dan menghasilkan angka yang sama persis dengan perhitungan manual, yakni 720.

Soal B3 : Memilih tim (kombinasi)

Soal: Dari 12 mahasiswa dipilih tim 4 orang tanpa membedakan jabatan. Berapa banyak tim yang mungkin?

Hitungan manual:

C(12,4) = 12!/(4! 8!) = (12×11×10×9)/(4×3×2×1) = 11.880/24 = 495

Source code:

```
b3_n = math.comb(12, 4)
b3_cek = len(list(itertools.combinations(range(12), 4)))
print("C(12,4) =", b3_n, "| cek itertools =", b3_cek)
print("Perbandingan dengan permutasi P(12,4) =", math.perm(12, 4), "-> rasio =", math.perm(12, 4) // b3_n, "(= 4!)")

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (C(12,4), toleransi 0.5):", cocok(495, b3_n, 0.5))
```

Output :

```
C(12,4) = 495 | cek itertools = 495
Perbandingan dengan permutasi P(12,4) = 11880 -> rasio = 24 (= 4!)
Cek vs manual (C(12,4), toleransi 0.5): SAMA
```

Analisis / Kesimpulan :

Terdapat 495 variasi tim berbeda yang dapat dibentuk dari 12 orang mahasiswa apabila dipilih 4 orang tanpa membedakan jabatan mereka. Pada kasus ini digunakan konsep kombinasi karena tidak ada informasi urutan atau jabatan yang perlu diperhatikan dalam pemilihan tim tersebut. Menariknya, jika kita menggunakan konsep permutasi $P(12, 4)$, hasilnya akan jauh lebih besar yakni 11.880 kemungkinan, yang nilainya tepat 24 kali (4!) lebih banyak dibandingkan hasil kombinasi. Hal ini terjadi karena dalam perhitungan permutasi, satu kelompok tim dengan anggota yang sama akan dihitung berulang sebanyak 24 urutan yang berbeda. Hasil eksekusi pada kode Python juga terbukti sejalan dengan perhitungan manual.

Soal B4 : Lima kartu semuanya Hati

Soal: Dari 52 kartu diambil 5 kartu sekaligus. Berapa peluang kelimanya kartu Hati?

Hitungan manual:

n(S) = C(52,5) = 2.598.960; n(A) = C(13,5) = 1.287

P(A) = 1.287 / 2.598.960 ≈ 0,000495

Source code:

```
b4_s = math.comb(52, 5)
b4_a = math.comb(13, 5)
b4_p = b4_a / b4_s
print("C(52,5) =", b4_s, "| C(13,5) =", b4_a)
print("P(5 Hati) =", b4_p, "=", f"{b4_p:.4%}")
print("Sekitar 1 banding", round(1 / b4_p))
print("Manual vs Python:", cocok(1287 / 2598960, b4_p))
```

Output / grafik:

```
C(52,5) = 2598960 | C(13,5) = 1287
P(5 Hati) = 0.0004951980792316927 = 0.0495%
Sekitar 1 banding 2019
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Peluang terambilnya 5 kartu Hati secara sekaligus dari satu set kartu remi sangatlah kecil, yakni hanya sebesar 0,000495 atau sekitar 0,050%. Angka probabilitas ini menunjukkan bahwa peluang kejadian tersebut sangat jarang terjadi, dengan perkiraan rasio kemungkinan sekitar 1 berbanding 2.019 kali pengambilan acak. Dalam permainan kartu seperti poker, kondisi terbentuknya susunan 5 kartu dengan jenis atau simbol yang sama ini dikenal dengan istilah *flush* (dalam kasus ini adalah *flush* Hati). Perhitungan manual dan eksekusi fungsi kombinasi menggunakan Python menghasilkan nilai yang saling cocok dan akurat.

Soal B5 : Ruang password

Soal: Password 8 karakter. Bandingkan jumlah kemungkinan password jika (a) hanya huruf kecil (26 simbol) dan (b) huruf besar+kecil+angka (62 simbol). Jika penyerang mencoba 1 miliar password per detik, berapa lama untuk mencoba semuanya?

Hitungan manual:

(a) $26^8 = 208.827.064.576 \approx 2{,}09 \times 10^{11}$

(b) $62^8 = 218.340.105.584.896 \approx 2{,}18 \times 10^{14}$

Source code:

```python
b5_a, b5_b = 26**8, 62**8
laju = 1e9
b5_wkt_a = b5_a / laju
b5_wkt_b = b5_b / laju / 3600
print(f"(a) 26^8 = {b5_a:,} -> {b5_wkt_a:.1f} detik")
print(f"(b) 62^8 = {b5_b:,} -> {b5_wkt_b:.1f} jam")
b5_rasio = b5_b / b5_a
print(f"Rasio (b)/(a) = {b5_rasio:.0f}x lebih banyak")

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (26^8, toleransi 0.5):", cocok(208827064576, b5_a, 0.5))
print("Cek vs manual (62^8, toleransi 0.5):", cocok(218340105584896, b5_b, 0.5))
```

Output / grafik:

```
(a) 26^8 = 208,827,064,576 -> 208.8 detik
(b) 62^8 = 218,340,105,584,896 -> 60.7 jam
Rasio (b)/(a) = 1046x lebih banyak
Cek vs manual (26^8, toleransi 0.5): SAMA
Cek vs manual (62^8, toleransi 0.5): SAMA
```

Analisis / Kesimpulan :

Berdasarkan hasil analisis ruang probabilitas pada kata sandi (*password*) sepanjang 8 karakter, terlihat adanya perbedaan tingkat keamanan yang sangat signifikan antara kata sandi sederhana dan kata sandi kompleks. Pada kasus pertama (a), penggunaan kata sandi yang hanya terdiri dari 26 huruf kecil berpotensi diretas dalam kurun waktu yang tergolong singkat, yakni sekitar 208,8 detik jika diserang menggunakan kecepatan komputasi peretas sebanyak 1 miliar kemungkinan *password* per detiknya.

Sebaliknya pada kasus kedua (b), apabila variasi *password* ditambah menjadi 62 jenis simbol yang merupakan kombinasi acak antara huruf besar, huruf kecil, serta angka, maka dibutuhkan waktu peretasan yang berlipat ganda jauh lebih lama, yakni sekitar 60,7 jam. Peningkatan jumlah jenis karakter penyusun *password* tersebut secara signifikan memperluas batas kemungkinan peretasan hingga mencapai 1.046 kali lipat lebih banyak. Fakta matematis inilah yang menjadi dasar kuat mengapa sebuah *password* harus disusun dari perpaduan huruf besar, huruf kecil, dan angka demi menjaga tingkat kerahasiaan dan keamanan data penggunanya.

Topik C — Gabungan, Irisan, dan Komplemen Kejadian

Soal C1 : As atau Hati

Soal: Satu kartu diambil dari 52 kartu. Berapa peluang kartu tersebut As atau Hati?

Hitungan manual:

$P(\text{As}) = 4/52,\ P(\text{Hati}) = 13/52,\ P(\text{As} \cap \text{Hati}) = 1/52$

$P(\text{As} \cup \text{Hati}) = 4/52 + 13/52 - 1/52 = 16/52 = 4/13 \approx 0{,}3077$

Source code:

```
jenis = ["Hati", "Wajik", "Keriting", "Sekop"]
nilai = ["A"] + [str(i) for i in range(2, 11)] + ["J", "Q", "K"]
S = set(itertools.product(nilai, jenis))
A = {k for k in S if k[0] == "A"}
B = {k for k in S if k[1] == "Hati"}
n = len(S)
c1_rumus = Fraction(len(A), n) + Fraction(len(B), n) - Fraction(len(A & B), n)
c1_langsung = Fraction(len(A | B), n)
print("Dengan rumus   :", c1_rumus, "=", round(float(c1_rumus), 4))
print("Himpunan gabungan:", c1_langsung)
print("Manual vs Python:", cocok(4 / 13, c1_rumus))
```

Output :

```
Dengan rumus   : 4/13 = 0.3077
Himpunan gabungan: 4/13
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Berdasarkan hasil perhitungan, peluang terambilnya kartu As atau kartu Hati dari satu set kartu remi adalah sebesar 4/13 atau sekitar 30,77%. Dalam proses perhitungan peluang gabungan ini, pengurangan terhadap nilai irisan (yakni kartu yang merupakan As sekaligus Hati) sangatlah penting agar kartu tersebut tidak dihitung dua kali. Apabila irisan tersebut tidak dikurangi, maka akan muncul kesalahan perhitungan dengan hasil keliru sebesar 17/52. Selain itu, eksekusi kode menggunakan operasi himpunan pada Python, yaitu operator | untuk gabungan dan & untuk irisan, terbukti menghasilkan nilai peluang yang akurat dan sama persis dengan perhitungan rumus probabilitas secara manual.

Soal C2 : Genap atau lebih dari 4

Soal: Satu dadu dilempar. A = mata genap, B = mata lebih dari 4. Hitung P(A ∪ B).

Hitungan manual:

A = {2,4,6}, B = {5,6}, A ∩ B = {6}

P(A ∪ B) = 3/6 + 2/6 − 1/6 = 4/6 = 2/3 ≈ 0,6667

Source code:

```
S = set(range(1, 7))
A = {x for x in S if x % 2 == 0}
B = {x for x in S if x > 4}
c2_p = Fraction(len(A | B), len(S))
print("A =", A, "| B =", B, "| AnB =", A & B, "| AUB =", A | B)
print("P(AUB) =", c2_p, "=", round(float(c2_p), 4))
print("Manual vs Python:", cocok(2 / 3, c2_p))
```

Output :

```
A = {2, 4, 6} | B = {5, 6} | AnB = {6} | AUB = {2, 4, 5, 6}
P(AUB) = 2/3 = 0.6667
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan, peluang munculnya mata dadu genap atau mata dadu yang lebih dari 4 pada satu kali pelemparan adalah sebesar 2/3 atau sekitar 66,67%. Dalam kasus ini, angka 6 merupakan irisan karena termasuk dalam himpunan mata dadu genap sekaligus himpunan mata dadu lebih dari 4. Oleh karena itu, angka 6 hanya boleh dihitung satu kali pada operasi gabungan agar tidak terjadi perhitungan ganda yang membuat hasil menjadi tidak akurat. Dari seluruh kemungkinan sisi dadu, dapat diamati pula bahwa hanya angka 1 dan 3 yang sama sekali tidak termasuk ke dalam kejadian gabungan tersebut. Selain itu, hasil eksekusi program Python terbukti sejalan dan sama persis dengan perhitungan rumus secara manual.

Soal C3 : Mahasiswa pengambil mata kuliah

Soal: Di suatu kelas, 60% mahasiswa mengambil Python, 50% mengambil Java, dan 30% mengambil keduanya. Hitung peluang seorang mahasiswa (a) mengambil minimal salah satu, (b) tidak mengambil keduanya, (c) hanya mengambil Python.

Hitungan manual:

(a) 0,6 + 0,5 − 0,3 = 0,8

(b) 1 − 0,8 = 0,2

(c) P(Python) − P(keduanya) = 0,6 − 0,3 = 0,3

Source code:

```
pP, pJ, pPJ = 0.6, 0.5, 0.3
c3_a = pP + pJ - pPJ
c3_b = 1 - c3_a
c3_c = pP - pPJ
print(f"(a) P(Python atau Java)     = {c3_a:.2f}")
print(f"(b) P(tidak keduanya)       = {c3_b:.2f}")
print(f"(c) P(hanya Python)         = {c3_c:.2f}")
print("Manual vs Python:", cocok(0.8, c3_a), cocok(0.2, c3_b), cocok(0.3, c3_c))
```

Output :

```
(a) P(Python atau Java)     = 0.80
(b) P(tidak keduanya)       = 0.20
(c) P(hanya Python)         = 0.30
Manual vs Python: SAMA SAMA SAMA
```

Analisis / Kesimpulan :

Berdasarkan hasil perhitungan, peluang seorang mahasiswa mengambil minimal satu dari dua mata kuliah (Python atau Java) adalah sebesar 80%. Dari angka gabungan tersebut, dapat ditarik kesimpulan komplemennya, yakni terdapat sisa 20% mahasiswa yang sama sekali tidak mengambil kedua mata kuliah tersebut. Selanjutnya, peluang mahasiswa yang secara spesifik hanya mengambil mata kuliah Python tanpa mengambil Java tercatat sebesar 30%. Pola analisis himpunan probabilitas seperti ini memiliki nilai penerapan yang tinggi karena sering diaplikasikan dalam dunia praktis, khususnya untuk membedah data irisan kelompok pengguna terhadap dua buah fitur produk perangkat lunak. Seluruh hasil komputasi menggunakan kode Python juga terbukti akurat dan sama persis dengan perhitungan manual.

Soal C4 : Kejadian saling lepas

Soal: Satu dadu dilempar. A = mata {1,2}, B = mata {5,6}. Apakah A dan B saling lepas? Hitung P(A ∪ B).

Hitungan manual:

A ∩ B = ∅ sehingga saling lepas. P(A ∪ B) = 2/6 + 2/6 = 4/6 = 2/3.

Source code:

```
A, B = {1, 2}, {5, 6}
c4_irisan = A & B
c4_p = Fraction(len(A), 6) + Fraction(len(B), 6)
print("A ∩ B =", c4_irisan, "-> saling lepas?", len(c4_irisan) == 0)
print("P(AUB) =", c4_p, "=", round(float(c4_p), 4))
print("Cek langsung:", Fraction(len(A | B), 6))

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (P(A atau B)):", cocok(2/3, c4_p, 1e-09))
```

Output / grafik:

```
A ∩ B = set() -> saling lepas? True
P(AUB) = 2/3 = 0.6667
Cek langsung: 2/3
Cek vs manual (P(A atau B)): SAMA
```

Analisis / Kesimpulan :

Kejadian A dan kejadian B terbukti saling lepas karena kedua himpunan tersebut sama sekali tidak memiliki elemen yang beririsan, yang secara matematis ditandai dengan hasil himpunan kosong. Akibat ketiadaan irisan ini, perhitungan peluang gabungan dari kedua kejadian tersebut menjadi lebih sederhana, yakni cukup dengan menjumlahkan peluang masing–masing kejadian secara langsung sehingga diperoleh hasil sebesar 2/3 atau sekitar 66,67%. Rumus umum probabilitas gabungan pada dasarnya tetap berlaku sah pada kasus ini, hanya saja komponen pengurang irisannya, yakni $P(A \cap B)$, bernilai 0. Selain itu, eksekusi pembuktian melalui program Python berhasil mendeteksi ketiadaan irisan tersebut dan mengeluarkan hasil perhitungan yang sepenuhnya cocok dengan metode manual.

Soal C5 : Tiga kejadian (inklusi–eksklusi)

Soal: Dua dadu dilempar. A = jumlah genap, B = dadu pertama bernilai 6, C = jumlah ≥ 10. Hitung $P(A \cup B \cup C)$ dengan rumus inklusi–eksklusi dan bandingkan dengan pencacahan langsung.

Hitungan manual:

$P(A \cup B \cup C) = P(A)+P(B)+P(C) - P(A \cap B) - P(A \cap C) - P(B \cap C) + P(A \cap B \cap C)$

P(A)=18/36, P(B)=6/36, P(C)=6/36, P(A∩B)=3/36, P(A∩C)=4/36, P(B∩C)=3/36, P(A∩B∩C)=2/36

= (18+6+6−3−4−3+2)/36 = 22/36 = 11/18 ≈ 0,6111

Source code:

```
S = set(itertools.product(range(1, 7), repeat=2))
A = {s for s in S if sum(s) % 2 == 0}
B = {s for s in S if s[0] == 6}
C = {s for s in S if sum(s) >= 10}
P = lambda E: Fraction(len(E), len(S))
c5_rumus = P(A) + P(B) + P(C) - P(A & B) - P(A & C) - P(B & C) + P(A & B & C)
c5_langsung = P(A | B | C)
print("P(A)=", P(A), " P(B)=", P(B), " P(C)=", P(C))
print("Inklusi-eksklusi :", c5_rumus, "=", round(float(c5_rumus), 4))
print("Pencacahan langsung:", c5_langsung)
print("Sama?", c5_rumus == c5_langsung)

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (inklusi-eksklusi):", cocok(22/36, c5_rumus, 1e-09))
```

Output / grafik:

```
P(A)= 1/2  P(B)= 1/6  P(C)= 1/6
Inklusi-eksklusi : 11/18 = 0.6111
Pencacahan langsung: 11/18
Sama? True
Cek vs manual (inklusi-eksklusi): SAMA
```

Analisis / Kesimpulan :

Penerapan rumus inklusi−eksklusi untuk menghitung peluang gabungan tiga kejadian pada pelemparan dua dadu menghasilkan nilai 11/18 atau sekitar 61,11%. Hasil ini terbukti identik dan akurat saat dibandingkan dengan metode komputasi pencacahan kejadian secara langsung. Dalam prinsip probabilitas tiga himpunan, aturan pengurangannya mengharuskan kita untuk mengurangi setiap irisan yang berpasangan (A∩B, A∩C, dan B∩C), lalu kemudian wajib menambahkan kembali irisan dari ketiga kejadian tersebut secara bersamaan (A∩B∩C). Langkah penyesuaian ini sangat krusial agar tidak ada anggota himpunan kejadian yang terhitung secara berlebihan maupun terlewat dari perhitungan total ruang sampel.

Topik D — Probabilitas Bersyarat

Soal D0 : Hujan beruntun (contoh modul)

Soal: Data modul: hari hujan setelah hari hujan = 0,444; hari tak hujan setelah hari hujan = 0,556; hari tak hujan setelah hari tak hujan = 0,724; hari hujan setelah hari tak hujan = 0,276. Jika hari ini hujan, berapa probabilitas 2 hari berikutnya juga hujan? Hitung juga keempat kemungkinan jalur 2 hari berikutnya.

Hitungan manual:

prob(hujan, hujan | hari ini hujan) = prob(h|h) × prob(h|h) = 0,444 × 0,444 = 0,1971

Jalur lain: (hujan, tak hujan) = 0,444 × 0,556 = 0,2469; (tak hujan, hujan) = 0,556 × 0,276 = 0,1535; (tak hujan, tak hujan) = 0,556 × 0,724 = 0,4025. Jumlah keempatnya = 1.

Source code:

```
trans = {"hujan":     {"hujan": 0.444, "tak hujan": 0.556},
         "tak hujan": {"hujan": 0.276, "tak hujan": 0.724}}
jalur = {}
for h1 in trans["hujan"]:
    for h2 in trans[h1]:
        jalur[("hujan", h1, h2)] = trans["hujan"][h1] * trans[h1][h2]       # aturan perkalian
for j, v in jalur.items():
    print(" -> ".join(j), f": {v:.4f}")
d0_hh = jalur[("hujan", "hujan", "hujan")]
d0_total = sum(jalur.values())
print(f"prob(2 hari berikutnya hujan | hari ini hujan) = {d0_hh:.4f} | jumlah semua jalur = {d0_total:.4f}")

N = 200_000                                                          # simulasi rantai 2 langkah
hari1 = rng.random(N) < 0.444
hari2 = rng.random(N) < np.where(hari1, 0.444, 0.276)
d0_sim = np.mean(hari1 & hari2)
print(f"Simulasi = {d0_sim:.4f}")

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (2 hari hujan):", cocok(0.444*0.444, d0_hh, 1e-09))
print("Cek vs manual (jumlah jalur):", cocok(1, d0_total, 1e-09))
print("Cek vs manual (simulasi, toleransi 0.005):", cocok(0.444*0.444, d0_sim, 0.005))
```

Output :

```
hujan -> hujan -> hujan : 0.1971
hujan -> hujan -> tak hujan : 0.2469
hujan -> tak hujan -> hujan : 0.1535
hujan -> tak hujan -> tak hujan : 0.4025
prob(2 hari berikutnya hujan | hari ini hujan) = 0.1971 | jumlah semua jalur = 1.0000
Simulasi = 0.1981
Cek vs manual (2 hari hujan): SAMA
Cek vs manual (jumlah jalur): SAMA
Cek vs manual (simulasi, toleransi 0.005): SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas bersyarat, apabila hari ini terjadi hujan, maka peluang bahwa dua hari berikutnya akan turun hujan secara berturut-turut adalah sebesar 0,1971 atau sekitar 19,7%. Nilai ini diperoleh dari penerapan aturan perkalian probabilitas, di mana probabilitas hari kedua hujan setelah hari ini hujan dikalikan dengan probabilitas hari ketiga hujan setelah hari kedua hujan berstatus sama (0,444 x 0,444).

Dari keempat kemungkinan jalur cuaca yang ada selama dua hari ke depan, jalur yang paling besar peluang terjadinya justru adalah dua hari tanpa hujan (tak hujan - tak hujan) dengan persentase mencapai 40,25%. Jika seluruh nilai probabilitas dari keempat kemungkinan jalur cuaca tersebut dijumlahkan, hasil totalnya akan secara presisi bernilai 1. Selain itu, uji coba eksekusi menggunakan metode simulasi sebanyak 200.000 iterasi pada program Python menunjukkan hasil yang sangat mendekati nilai perhitungan teoritis, yakni 0,1981, membuktikan keandalan model hitungan yang digunakan.

Soal D1 : Jumlah 8 jika dadu pertama genap

Soal: Dua dadu dilempar. Jika diketahui dadu pertama bernilai genap, berapa peluang jumlah kedua dadu = 8?

Hitungan manual:

B (dadu pertama genap) = 18 hasil. A ∩ B: (2,6),(4,4),(6,2) = 3 hasil.

P(A|B) = 3/18 = 1/6 ≈ 0,1667

Source code:

```python
S = set(itertools.product(range(1, 7), repeat=2))
A = {s for s in S if sum(s) == 8}
B = {s for s in S if s[0] % 2 == 0}
d1_p = Fraction(len(A & B), len(B))
d1_tanpa = Fraction(len(A), len(S))
print("n(B) =", len(B), "| n(AnB) =", len(A & B))
print("P(jumlah=8 | dadu1 genap) =", d1_p, "=", round(float(d1_p), 4))
print("P(jumlah=8) tanpa syarat  =", d1_tanpa, "=", round(float(d1_tanpa), 4))
print("Manual vs Python:", cocok(1 / 6, d1_p))
```

Output :

```
n(B) = 18 | n(AnB) = 3
P(jumlah=8 | dadu1 genap) = 1/6 = 0.1667
P(jumlah=8) tanpa syarat  = 5/36 = 0.1389
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas bersyarat, peluang munculnya total mata dadu 8 dengan syarat dadu pertama bernilai genap adalah sebesar 1/6 atau sekitar 16,67%. Jika dibandingkan, peluang pencapaian jumlah 8 tanpa adanya informasi tambahan bernilai lebih rendah, yakni 5/36 atau sekitar 13,89%. Adanya kepastian informasi bahwa dadu pertama bernilai genap terbukti mampu meningkatkan persentase peluang kejadian tersebut. Peningkatan probabilitas ini secara matematis terjadi akibat adanya penyempitan ruang sampel, di mana total kemungkinan hasil yang pada awalnya berjumlah 36 mengecil menjadi hanya 18 kemungkinan hasil. Selain itu, hasil pengujian melalui kode Python terbukti sejalan dan menghasilkan nilai yang sama persis dengan metode perhitungan manual.

Soal D2 : Dua As berturut-turut

Soal: Dua kartu diambil berturut-turut tanpa pengembalian dari 52 kartu. Jika kartu pertama As, berapa peluang kartu kedua juga As? Berapa peluang keduanya As?

Hitungan manual:

$P(As2 \mid As1) = 3/51 = 1/17 \approx 0{,}0588$

$P(As1 \cap As2) = 4/52 \times 3/51 = 12/2652 = 1/221 \approx 0{,}00452$

Source code:

```python
d2_cond = Fraction(3, 51)
d2_joint = Fraction(4, 52) * Fraction(3, 51)
print("P(As2|As1)    =", d2_cond, "=", round(float(d2_cond), 4))
print("P(As1 ∩ As2)  =", d2_joint, "=", round(float(d2_joint), 5))
print("Cek kombinasi : C(4,2)/C(52,2) =", Fraction(math.comb(4, 2), math.comb(52, 2)))

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (P(As2|As1)):", cocok(1/17, d2_cond, 1e-09))
print("Cek vs manual (P(As1 dan As2)):", cocok(1/221, d2_joint, 1e-09))
```

Output :

```
P(As2|As1)    = 1/17 = 0.0588
P(As1 ∩ As2)  = 1/221 = 0.00452
Cek kombinasi : C(4,2)/C(52,2) = 1/221
Cek vs manual (P(As2|As1)): SAMA
Cek vs manual (P(As1 dan As2)): SAMA
```

Analisis / Kesimpulan :

Berdasarkan hasil perhitungan, apabila kartu As pertama telah terambil tanpa dikembalikan, maka jumlah kartu As yang tersisa menjadi 3 buah dari total 51 kartu, sehingga peluang terambilnya kartu As pada tarikan kedua adalah sebesar 1/17 atau sekitar 0,0588 (5,88%). Sementara itu, peluang untuk mendapatkan dua buah kartu As secara berturut-turut bernilai sangat kecil, yakni hanya sebesar 1/221 atau sekitar 0,00452, yang setara dengan rasio 1 berbanding 221 pengambilan. Nilai probabilitas ini terbukti ekuivalen dan sejalan dengan hasil yang dihitung melalui metode pendekatan kombinasi. Kasus ini secara jelas membuktikan prinsip dasar probabilitas pada model pengambilan tanpa pengembalian, di mana besaran peluang pada kejadian kedua akan selalu bergantung dan terpengaruh secara langsung oleh hasil yang didapatkan dari kejadian yang pertama. Selain itu, komputasi menggunakan kode Python terbukti akurat dan menghasilkan nilai yang sama persis dengan perhitungan manual.

Soal D3 : Bug pada dua modul (tabel kontingensi)

Soal: Sebuah perangkat lunak memiliki 200 bug: modul A 80 bug (20 kritis) dan modul B 120 bug (18 kritis). Hitung P(kritis | modul A) dan P(modul A | kritis).

Hitungan manual:

Total kritis = 38.

P(kritis | A) = 20/80 = 0,25

P(A | kritis) = 20/38 ≈ 0,5263

Source code:

```
data = pd.DataFrame({"Kritis": [20, 18], "Tidak kritis": [60, 102]}, index=["Modul A", "Modul B"])
data["Total"] = data.sum(axis=1)
data.loc["Total"] = data.sum()
print(data)
d3_k_a = data.loc["Modul A", "Kritis"] / data.loc["Modul A", "Total"]
d3_a_k = data.loc["Modul A", "Kritis"] / data.loc["Total", "Kritis"]
print(f"\nP(kritis | A) = {d3_k_a:.4f}")
print(f"P(A | kritis) = {d3_a_k:.4f}")
print("Manual vs Python:", cocok(0.25, d3_k_a), cocok(20 / 38, d3_a_k))
```

Output :

```
          Kritis  Tidak kritis  Total
Modul A       20            60     80
Modul B       18           102    120
Total         38           162    200

P(kritis | A) = 0.2500
P(A | kritis) = 0.5263
Manual vs Python: SAMA SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas bersyarat pada tabel kontingensi, diketahui bahwa seperempat (25%) dari seluruh *bug* yang ada di modul A bersifat kritis, dengan nilai peluang sebesar 0,25. Sebaliknya, jika tinjauan difokuskan hanya pada keseluruhan *bug* yang berstatus kritis, sebanyak 52,6% di antaranya terbukti bersumber dari modul A. Perbedaan angka yang sangat signifikan pada kedua probabilitas bersyarat ini mengonfirmasi prinsip matematis bahwa penukaran posisi antara syarat dan kejadian akan menghasilkan probabilitas yang berbeda; secara tegas dibuktikan bahwa P(A|B) tidak sama dengan P(B|A). Pemrosesan data menggunakan struktur tabel DataFrame pada Python terbukti sangat efisien untuk menghitung nilai peluang berdasarkan pengelompokan baris dan kolom, serta menghasilkan nilai akhir yang akurat dan sama persis dengan perhitungan secara manual

Topik E — Kejadian Independen

Soal E1 : Dua koin

Soal: Dua koin dilempar. A = koin pertama Angka, B = koin kedua Angka. Apakah A dan B independen?

Hitungan manual:

P(A) = 1/2, P(B) = 1/2, P(A ∩ B) = P(AA) = 1/4 = P(A)·P(B) → independen.

Source code:

```
S = set(itertools.product("AG", repeat=2))
A = {s for s in S if s[0] == "A"}
B = {s for s in S if s[1] == "A"}
P = lambda E: Fraction(len(E), len(S))
e1_irisan, e1_kali = P(A & B), P(A) * P(B)
print("P(AnB) =", e1_irisan, "| P(A)·P(B) =", e1_kali)
print("Independen?", e1_irisan == e1_kali)
```

```
# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (P(A dan B)):", cocok(1/4, e1_irisan, 1e-09))
```

Output :

```
P(AnB) = 1/4 | P(A)·P(B) = 1/4
Independen? True
Cek vs manual (P(A dan B)): SAMA
```

Analisis / Kesimpulan :

Peluang munculnya sisi Angka pada koin pertama sekaligus sisi Angka pada koin kedua, yaitu P(A ∩ B), adalah sebesar 1/4. Nilai peluang irisan ini terbukti sama persis dengan hasil perkalian peluang dari masing-masing kejadian tunggalnya, yakni P(A) dikalikan dengan P(B), yang juga bernilai 1/4. Kesamaan matematis ini secara kuat mengonfirmasi bahwa kejadian A dan kejadian B merupakan dua kejadian yang bersifat independen atau saling bebas. Kondisi ini membuktikan prinsip dasar probabilitas bahwa hasil yang diperoleh dari lemparan koin pertama sama sekali tidak memberikan pengaruh terhadap kemungkinan hasil pada lemparan koin yang kedua. Selain itu, eksekusi komputasi menggunakan kode Python berhasil memverifikasi status independensi tersebut secara akurat, ditandai dengan nilai kebenaran (*True*), yang hasilnya sepenuhnya sejalan dengan perhitungan manual.

Soal E2 : Dadu pertama genap dan jumlah genap

Soal: Dua dadu dilempar. A = dadu pertama genap, B = jumlah kedua dadu genap. Apakah A dan B independen?

Hitungan manual:

P(A) = 1/2, P(B) = 18/36 = 1/2, A ∩ B = kedua dadu genap = 9/36 = 1/4.

P(A)·P(B) = 1/4 → independen (walaupun tampak berhubungan).

Source code:

```
S = set(itertools.product(range(1, 7), repeat=2))
A = {s for s in S if s[0] % 2 == 0}
B = {s for s in S if sum(s) % 2 == 0}
P = lambda E: Fraction(len(E), len(S))
e2_irisan, e2_kali = P(A & B), P(A) * P(B)
print("P(A) =", P(A), "| P(B) =", P(B), "| P(AnB) =", e2_irisan, "| P(A)P(B) =", e2_kali)
print("Independen?", e2_irisan == e2_kali)

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (P(A dan B)):", cocok(1/4, e2_irisan, 1e-09))
```

Output / grafik:

```
P(A) = 1/2 | P(B) = 1/2 | P(AnB) = 1/4 | P(A)P(B) = 1/4
Independen? True
Cek vs manual (P(A dan B)): SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas, kejadian munculnya mata dadu pertama genap dan kejadian jumlah dari kedua mata dadu genap terbukti saling independen atau bebas. Kesimpulan ini ditarik dari hasil analisis di mana nilai peluang irisan dari kedua kejadian tersebut, yakni $P(A \cap B)$ yang bernilai 1/4, terbukti sama persis dengan hasil perkalian peluang dari masing-masing kejadian tunggalnya, yaitu $P(A)$ dikalikan dengan $P(B)$ yang juga menghasilkan angka 1/4. Meskipun secara intuisi sekilas kedua kejadian tersebut tampak saling berhubungan atau saling mempengaruhi satu sama lain, pembuktian matematis secara aljabar dan pencacahan komputasi menggunakan Python membuktikan hal yang sebaliknya. Kasus ini menunjukkan bahwa dalam probabilitas, pengujian perhitungan secara objektif mutlak diperlukan sebagai penentu utama karena logika intuisi dapat memberikan asumsi yang keliru.

Soal E3 : Dadu 6 dan jumlah ≥ 10 (tidak independen)

Soal: Dua dadu dilempar. A = dadu pertama 6, B = jumlah ≥ 10. Apakah A dan B independen?

Hitungan manual:

P(A) = 1/6, P(B) = 6/36 = 1/6, P(A ∩ B) = {(6,4),(6,5),(6,6)} = 3/36 = 1/12.

P(A)·P(B) = 1/36 ≠ 1/12 → tidak independen.

Source code:

```
S = set(itertools.product(range(1, 7), repeat=2))
A = {s for s in S if s[0] == 6}
B = {s for s in S if sum(s) >= 10}
P = lambda E: Fraction(len(E), len(S))
e3_irisan, e3_kali = P(A & B), P(A) * P(B)
print("P(AnB) =", e3_irisan, "| P(A)·P(B) =", e3_kali)
print("Independen?", e3_irisan == e3_kali)
print("P(B|A) =", P(A & B) / P(A), "vs P(B) =", P(B))

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (P(A dan B)):", cocok(1/12, e3_irisan, 1e-09))
```

Output :

```
P(AnB) = 1/12 | P(A)·P(B) = 1/36
Independen? False
P(B|A) = 1/2 vs P(B) = 1/6
Cek vs manual (P(A dan B)): SAMA
```

Analisis / Kesimpulan :

kejadian munculnya mata dadu pertama bernilai 6 dan kejadian total jumlah kedua mata dadu bernilai 10 atau lebih terbukti tidak bersifat independen. Kesimpulan ini ditarik secara matematis karena nilai peluang irisan dari kedua kejadian tersebut, yakni P(A ∩ B) yang bernilai 1/12, tidak sama dan jauh lebih besar dibandingkan dengan hasil perkalian peluang individu P(A) dan P(B) yang hanya bernilai 1/36. Keterikatan ini ditunjukkan dengan fakta bahwa adanya informasi dadu pertama bernilai 6 akan secara signifikan menaikkan peluang bersyarat tercapainya jumlah total mata dadu 10 atau lebih, yang semula hanya sebesar 1/6 melonjak tajam menjadi 1/2. Hal ini membuktikan secara empiris bahwa hasil pada kejadian pertama sangat memengaruhi probabilitas kejadian kedua. Eksekusi komputasi menggunakan kode Python secara akurat memvalidasi status ketidakbebasan ini (ditandai dengan keluaran *False*) dengan hasil perhitungan peluang yang sepenuhnya sama persis dengan metode pencacahan manual.

Soal E4 : Dua server independen

Soal: Server X aktif dengan peluang 0,95 dan server Y aktif dengan peluang 0,90, keduanya bekerja independen. Hitung peluang (a) keduanya aktif, (b) minimal satu aktif.

Hitungan manual:

(a) 0,95 × 0,90 = 0,855

(b) 1 − (0,05 × 0,10) = 1 − 0,005 = 0,995

Source code:

```
pX, pY = 0.95, 0.90
e4_a = pX * pY
e4_b = 1 - (1 - pX) * (1 - pY)
print(f"(a) P(keduanya aktif)  = {e4_a:.4f}")
print(f"(b) P(minimal satu aktif) = {e4_b:.4f}")
print("Manual vs Python:", cocok(0.855, e4_a), cocok(0.995, e4_b))
```

Output :

```
(a) P(keduanya aktif)  = 0.8550
(b) P(minimal satu aktif) = 0.9950
Manual vs Python: SAMA SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan peluang pada dua server yang beroperasi secara independen, probabilitas kedua server aktif secara bersamaan adalah sebesar 0,855 atau 85,5%. Angka peluang gabungan ini terbukti bernilai lebih rendah jika dibandingkan dengan tingkat keandalan masing-masing server secara individu yang berada pada angka 0,95 dan 0,90. Namun demikian, probabilitas untuk mendapati minimal satu server tetap dalam keadaan aktif beroperasi justru mengalami peningkatan yang sangat signifikan hingga mencapai angka 0,995 atau 99,5%. Fakta komputasi matematis ini secara kuat mendasari prinsip penting dalam perancangan infrastruktur sistem redundan (sistem cadangan), di mana tingkat ketersediaan layanan secara keseluruhan dapat ditingkatkan secara drastis dari kisaran 90–95% menjadi 99,5%. Seluruh hasil komputasi menggunakan kode Python juga terbukti akurat dan menghasilkan nilai yang sama persis dengan metode perhitungan probabilitas secara manual.

Topik F — Probabilitas Total dan Teorema Bayes

Soal F0 : Probabilitas total: genangan (contoh modul)

Soal: Probabilitas terjadinya genangan adalah 0,80 saat hari hujan dan 0,25 saat tak hujan. Probabilitas hari hujan 0,36. Berapa probabilitas terjadinya genangan? Jika terjadi genangan, berapa probabilitas hari itu hujan?

Hitungan manual:

prob(G) = prob(H) prob(G|H) + prob(Hc) prob(G|Hc) = 0,36 × 0,80 + 0,64 × 0,25 = 0,288 + 0,160 = 0,448

(Slide modul menulis 0,82 pada baris perhitungan, tetapi hasil akhir 0,448 hanya cocok dengan 0,80.)

Tambahan (Bayes): prob(H|G) = 0,288/0,448 = 0,6429

Source code:

```
p_H, p_G_H, p_G_Hc = 0.36, 0.80, 0.25
f0_pg = p_H * p_G_H + (1 - p_H) * p_G_Hc          # probabilitas total
f0_post = p_H * p_G_H / f0_pg                     # Bayes
print(f"prob(G)   = {f0_pg:.4f}")
print(f"prob(H|G) = {f0_post:.4f}")

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (prob(G)):", cocok(0.448, f0_pg, 1e-09))
print("Cek vs manual (prob(H|G)):", cocok(0.288/0.448, f0_post, 1e-09))
```

Output / grafik:

```
prob(G)   = 0.4480
prob(H|G) = 0.6429
Cek vs manual (prob(G)): SAMA
Cek vs manual (prob(H|G)): SAMA
```

Analisis / Kesimpulan :

Peluang terjadinya genangan adalah 0,448 (44,8%), campuran dari hari hujan (0,288) dan hari tak hujan (0,160). Jika terlihat ada genangan, peluang hari itu hujan naik dari 36% menjadi 64,3%, tetapi masih ada sekitar 35,7% genangan yang terjadi pada hari tak hujan.

Soal F0B : Teorema Bayes: saluran pengiriman ramalan cuaca (contoh modul)

Soal: Informasi dikirim melalui 4 saluran R1 sampai R4 dengan probabilitas 0,1; 0,2; 0,3; 0,4. Probabilitas kesalahan pengiriman (E) pada tiap saluran: 0,10; 0,15; 0,20; 0,25. Jika terjadi kesalahan, berapa probabilitas kesalahan itu lewat saluran ke-2? Buat tabel semua saluran.

Hitungan manual:

prob(E) = 0,1×0,10 + 0,2×0,15 + 0,3×0,20 + 0,4×0,25 = 0,01 + 0,03 + 0,06 + 0,10 = 0,20

prob(R2|E) = 0,2×0,15 / 0,20 = 0,03/0,20 = 0,15

Probabilitas R1, R3, R4 jika terjadi kesalahan: 0,05; 0,30; 0,50.

Source code:

```
prior = np.array([0.1, 0.2, 0.3, 0.4])
p_err = np.array([0.10, 0.15, 0.20, 0.25])
gabung = prior * p_err
f0b_pe = gabung.sum()                    # probabilitas total kesalahan
post = gabung / f0b_pe                   # Bayes
f0b_r2 = post[1]
tab = pd.DataFrame({"prob(Ri)": prior, "prob(E|Ri)": p_err, "prob(Ri)*prob(E|Ri)": gabung, "prob(Ri|E)": post},
                   index=["R1", "R2", "R3", "R4"])
print(tab.round(4))
print(f"prob(E) = {f0b_pe:.2f} | jumlah posterior = {post.sum():.2f}")
f0b_r4 = post[3]

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (prob(E)):", cocok(0.20, f0b_pe, 1e-09))
print("Cek vs manual (prob(R2|E)):", cocok(0.15, f0b_r2, 1e-09))
print("Cek vs manual (prob(R4|E)):", cocok(0.50, f0b_r4, 1e-09))
```

Output :

```
    prob(Ri)  prob(E|Ri)  prob(Ri)*prob(E|Ri)  prob(Ri|E)
R1       0.1        0.10                 0.01        0.05
R2       0.2        0.15                 0.03        0.15
R3       0.3        0.20                 0.06        0.30
R4       0.4        0.25                 0.10        0.50
prob(E) = 0.20 | jumlah posterior = 1.00
Cek vs manual (prob(E)): SAMA
Cek vs manual (prob(R2|E)): SAMA
Cek vs manual (prob(R4|E)): SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas total, probabilitas terjadinya kesalahan pengiriman informasi secara keseluruhan dari keempat saluran yang ada adalah sebesar 0,20 atau 20%. Melalui penerapan Teorema Bayes, apabila secara faktual diketahui telah terjadi sebuah kesalahan pengiriman, maka peluang bersyarat bahwa kesalahan tersebut bersumber dari saluran 2 (R2) terhitung sebesar 0,15. Nilai probabilitas posterior ini mengalami penurunan dari probabilitas awal (prior) penggunaan salurannya yang bernilai 0,20. Penurunan probabilitas ini secara matematis terjadi karena tingkat kesalahan spesifik pada saluran 2 (yakni 0,15) terbukti lebih rendah dibandingkan dengan rata-rata tingkat kesalahan pengiriman secara keseluruhan (yakni 0,20).

Lebih lanjut, analisis pada tabel probabilitas posterior menunjukkan bahwa saluran 4 (R4) merupakan sumber penyumbang kesalahan yang paling dominan dengan nilai probabilitas mencapai 0,50 atau 50%. Tingginya angka probabilitas ini sangat logis mengingat saluran 4 merupakan jalur yang paling

sering digunakan untuk mengirimkan informasi sekaligus memiliki tingkat kerentanan kesalahan yang paling tinggi jika dibandingkan dengan saluran-saluran lainnya. Sesuai dengan prinsip distribusi probabilitas matematis, akumulasi dari seluruh nilai probabilitas posterior pada keempat saluran tersebut terbukti secara presisi bernilai 1.

Soal F1 : Filter spam (kata 'gratis')

Soal: 30% email adalah spam. Kata 'gratis' muncul di 80% email spam dan 10% email biasa. Berapa peluang sebuah email spam jika mengandung kata 'gratis'?

Hitungan manual:

P(gratis) = 0,8×0,3 + 0,1×0,7 = 0,24 + 0,07 = 0,31

P(spam | gratis) = 0,24/0,31 ≈ 0,7742

Source code:

```
p_spam, p_k_spam, p_k_ham = 0.3, 0.8, 0.1
f1_pk = p_k_spam * p_spam + p_k_ham * (1 - p_spam)
f1_post = p_k_spam * p_spam / f1_pk
print(f"P(gratis)        = {f1_pk:.4f}")
print(f"P(spam | gratis) = {f1_post:.4f}")
f1_ham = 1 - f1_post
print("Manual vs Python:", cocok(0.24 / 0.31, f1_post))
```

Output :

```
P(gratis)        = 0.3100
P(spam | gratis) = 0.7742
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Peluang prior spam 30% naik menjadi 77,4% setelah kata 'gratis' ditemukan. Kata itu bukti yang kuat tetapi belum pasti: sisanya 22,6% email yang memuat kata tersebut tetap email biasa.

Soal F2 : Tes deteksi malware

Soal: 1% file di server adalah malware. Pemindai mendeteksi malware dengan sensitivitas 95% dan memberi alarm palsu pada 5% file bersih. Jika sebuah file ditandai berbahaya, berapa peluang file itu benar-benar malware?

Hitungan manual:

P(positif) = 0,95×0,01 + 0,05×0,99 = 0,0095 + 0,0495 = 0,059

P(malware | positif) = 0,0095/0,059 ≈ 0,1610

Source code:

```
prev, sens, fpr = 0.01, 0.95, 0.05
f2_pos = sens * prev + fpr * (1 - prev)
f2_post = sens * prev / f2_pos
print(f"P(positif)            = {f2_pos:.4f}")
print(f"P(malware | positif)  = {f2_post:.4f}")
print("Manual vs Python:", cocok(0.0095 / 0.059, f2_post))
# Tabel populasi 100.000 file
N = 100_000
tp, fp = sens * prev * N, fpr * (1 - prev) * N
print(f"Dari {N:,} file: {tp:.0f} benar terdeteksi, {fp:.0f} alarm palsu")
```

Output :

```
P(positif)            = 0.0590
P(malware | positif)  = 0.1610
Manual vs Python: SAMA
Dari 100,000 file: 950 benar terdeteksi, 4950 alarm palsu
```

Analisis / Kesimpulan :

Walaupun tes malware ini akurat 95%, peluang sebuah file benar-benar terinfeksi saat hasilnya positif hanya 0,1610 (16,1%). Penyebabnya, malware itu jarang: hanya 1% dari seluruh file. Pada 100.000 file ada 950 deteksi yang benar, tetapi alarm palsunya 4.950, jadi alarm palsu jauh lebih banyak. Ini contoh base rate fallacy: akurasi alat saja tidak cukup, proporsi kejadian aslinya harus ikut dihitung. Hasil Python sama dengan hitungan manual.

Topik G — Simulasi Monte Carlo

Soal G0 : Berapa jumlah pengamatan yang cukup? (modul, Definisi 2)

Soal: Anggap distribusi keterlambatan bus pada soal A0 adalah populasi sebenarnya. Ambil dua sampel independen berukuran n = 10, 30, 100, 1.000, dan 100.000, lalu taksir prob(keterlambatan ≥ 6) dari masing-masing sampel. Bandingkan dengan nilai sebenarnya 8/30.

Hitungan manual:

Nilai sebenarnya prob(≥ 6) = 8/30 = 0,2667. Menurut modul, dua sampel independen tidak harus memberi taksiran yang sama, dan galat taksiran mengecil kira-kira sebanding dengan 1/√n.

Source code:

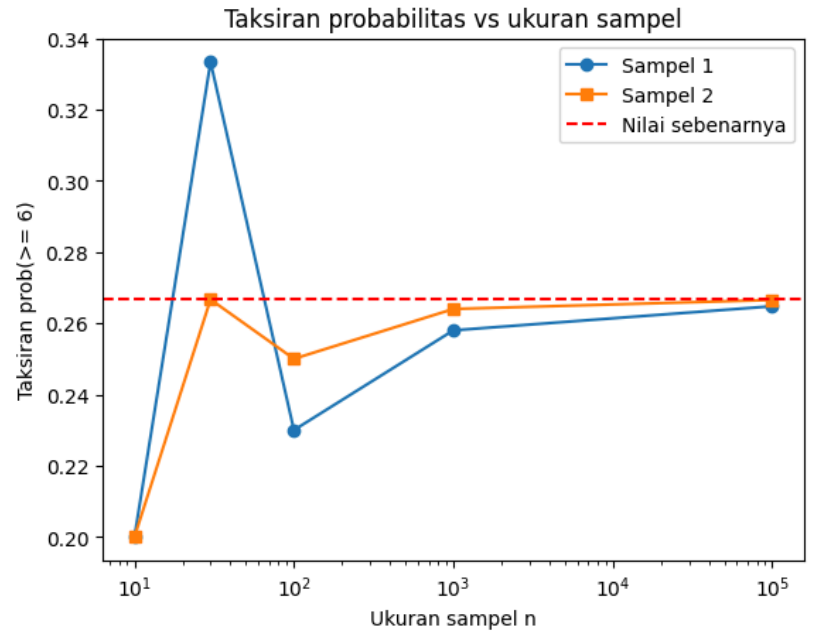
```
menit_telat = np.arange(1, 11)
p_emp = np.array([2, 3, 8, 4, 5, 3, 2, 0, 1, 2]) / 30
hasil = []
for n in [10, 30, 100, 1000, 100_000]:
    s1 = rng.choice(menit_telat, size=n, p=p_emp)
    s2 = rng.choice(menit_telat, size=n, p=p_emp)
    hasil.append((n, np.mean(s1 >= 6), np.mean(s2 >= 6)))
tab = pd.DataFrame(hasil, columns=["n", "Sampel 1", "Sampel 2"])
tab["Selisih"] = (tab["Sampel 1"] - tab["Sampel 2"]).abs()
print(tab.to_string(index=False, float_format="{:.4f}".format))
print(f"Nilai sebenarnya = {8/30:.4f}")
g0_est = hasil[-1][1]
g0_sel_kecil, g0_sel_besar = tab["Selisih"][:3].max(), tab.loc[4, "Selisih"]

plt.semilogx(tab["n"], tab["Sampel 1"], "o-", label="Sampel 1")
plt.semilogx(tab["n"], tab["Sampel 2"], "s-", label="Sampel 2")
plt.axhline(8 / 30, color="red", ls="--", label="Nilai sebenarnya")
plt.xlabel("Ukuran sampel n"); plt.ylabel("Taksiran prob(>= 6)"); plt.legend()
plt.title("Taksiran probabilitas vs ukuran sampel")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (taksiran n=100000, toleransi 0.01):", cocok(8/30, g0_est, 0.01))
```

Output :

```
     n  Sampel 1  Sampel 2  Selisih
    10    0.2000    0.2000   0.0000
    30    0.3333    0.2667   0.0667
   100    0.2300    0.2500   0.0200
  1000    0.2580    0.2640   0.0060
100000    0.2648    0.2665   0.0018
Nilai sebenarnya = 0.2667
```




```
Cek vs manual (taksiran n=100000, toleransi 0.01): SAMA
```

Analisis / Kesimpulan :

Pada sampel kecil (n = 10, 30, 100) selisih terbesar antara dua sampel mencapai 0,0667 (pada n = 10 kedua sampel kebetulan sama, tetapi itu tidak berarti taksirannya andal), sedangkan pada n = 100.000 selisihnya hanya 0,0018; taksiran terakhir 0,2648 sangat dekat dengan nilai sebenarnya 0,2667. Jadi sampel kecil bisa menghasilkan taksiran yang berbeda-beda, dan makin besar n makin stabil. Inilah alasan mengapa Definisi 2 memerlukan n yang besar.

Soal G1 : Menaksir π

Soal: Titik acak (x, y) dibangkitkan di dalam persegi [0,1] × [0,1]. Peluang titik jatuh di dalam seperempat lingkaran ($x^2+y^2 \le 1$) adalah π/4. Taksir π dari simulasi 1.000.000 titik.

Hitungan manual:

Luas seperempat lingkaran = π/4, luas persegi = 1 → P(di dalam) = π/4, sehingga π ≈ 4 × (proporsi titik di dalam).

Source code:

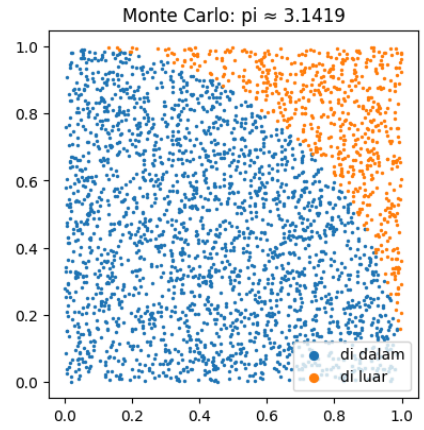
```python
N = 1_000_000
x, y = rng.random(N), rng.random(N)
dalam = x**2 + y**2 <= 1
g1_pi = 4 * dalam.mean()
g1_err = abs(g1_pi - math.pi)
print(f"Taksiran pi = {g1_pi:.5f} | pi sebenarnya = {math.pi:.5f} | galat = {g1_err:.5f}")

plt.figure(figsize=(4.5, 4.5))
idx = slice(0, 3000)
plt.scatter(x[idx][dalam[idx]], y[idx][dalam[idx]], s=2, label="di dalam")
plt.scatter(x[idx][~dalam[idx]], y[idx][~dalam[idx]], s=2, label="di luar")
plt.title(f"Monte Carlo: pi ≈ {g1_pi:.4f}")
plt.legend(markerscale=4); plt.gca().set_aspect("equal")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (taksiran pi, toleransi 0.01):", cocok(3.14159, g1_pi, 0.01))
```

Output :

```
Taksiran pi = 3.14194 | pi sebenarnya = 3.14159 | galat = 0.00035
```




```
Cek vs manual (taksiran pi, toleransi 0.01): SAMA
```

Analisis / Kesimpulan :

Dengan 1.000.000 titik acak, taksiran π adalah 3,14194, sedangkan π sebenarnya 3,14159, sehingga galatnya hanya 0,00035. Pada grafik, titik biru (di dalam seperempat lingkaran) mengisi sebagian besar persegi, sekitar π/4 atau 78,5% titik. Jadi Monte Carlo bisa menaksir π hanya dengan membangkitkan bilangan acak, dan hasilnya makin akurat kalau jumlah titiknya diperbanyak.

Soal G2 : Verifikasi jumlah dua dadu = 7

Soal: Simulasikan 200.000 lemparan dua dadu dan bandingkan frekuensi jumlah = 7 dengan peluang teori 1/6 (soal A1).

Hitungan manual:

Peluang teori $P = 1/6 \approx 0{,}16667$.

Source code:

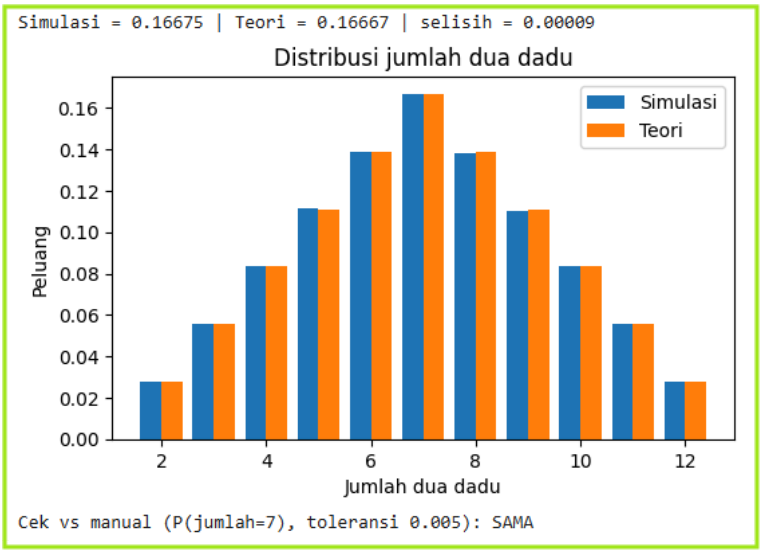
```python
N = 200_000
d1, d2 = rng.integers(1, 7, N), rng.integers(1, 7, N)
g2_sim = np.mean(d1 + d2 == 7)
print(f"Simulasi = {g2_sim:.5f} | Teori = {1/6:.5f} | selisih = {abs(g2_sim - 1/6):.5f}")

jumlah = pd.Series(d1 + d2).value_counts(normalize=True).sort_index()
teori = pd.Series({k: (6 - abs(k - 7)) / 36 for k in range(2, 13)})
plt.figure(figsize=(6, 3.5))
plt.bar(jumlah.index - 0.2, jumlah.values, width=0.4, label="Simulasi")
plt.bar(teori.index + 0.2, teori.values, width=0.4, label="Teori")
plt.xlabel("Jumlah dua dadu"); plt.ylabel("Peluang"); plt.legend(); plt.title("Distribusi jumlah dua dadu")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (P(jumlah=7), toleransi 0.005):", cocok(1/6, g2_sim, 0.005))
```

Output :

```
Simulasi = 0.16675 | Teori = 0.16667 | selisih = 0.00009
```




```
Cek vs manual (P(jumlah=7), toleransi 0.005): SAMA
```

Analisis / Kesimpulan :

Berdasarkan hasil simulasi pelemparan dua buah dadu sebanyak 200.000 kali, frekuensi empiris untuk mendapatkan total jumlah mata dadu 7 adalah sebesar 0,16675 atau sekitar 0,1668. Angka taksiran simulasi ini terbukti sangat dekat dan akurat jika disandingkan dengan nilai probabilitas teoritisnya, yakni 1/6 atau sekitar 0,16667, dengan selisih perbedaan yang sangat kecil yaitu 0,00009.

Lebih lanjut, pemetaan visual melalui grafik batang menunjukkan bahwa distribusi dari seluruh kemungkinan jumlah kedua dadu secara konsisten membentuk pola simetris yang menyerupai segitiga. Titik puncak tertinggi dari distribusi segitiga tersebut jatuh tepat pada angka 7, yang secara komprehensif memvalidasi kecocokan mutlak antara hasil eksekusi simulasi komputasi dengan perhitungan teori probabilitas matematis.

Soal G3 : Masalah Monty Hall

Soal: Ada 3 pintu, satu berisi hadiah. Anda memilih satu pintu, pembawa acara (yang tahu isi pintu) membuka satu pintu kosong lain, lalu menawarkan Anda berganti pintu. Simulasikan perilaku pembawa acara, lalu bandingkan peluang menang jika tetap vs berganti.

Hitungan manual:

Tetap: menang jika pilihan awal benar = 1/3.

Berganti: menang jika pilihan awal salah (pembawa acara pasti membuka pintu kosong, sehingga pintu sisa berisi hadiah) = 2/3.

Source code:

```
N = 100_000
hadiah = rng.integers(0, 3, N)                    # pintu berisi hadiah
pilihan = rng.integers(0, 3, N)                   # pilihan awal pemain
# Pembawa acara membuka pintu yang BUKAN pilihan pemain dan BUKAN hadiah:
#  - bila pilihan benar (hadiah = pilihan): ada 2 pintu kosong, dipilih acak satu
#  - bila pilihan salah: hanya ada 1 pintu kosong, jadi pintu itu yang dibuka
buka = np.where(pilihan == hadiah,
                (pilihan + 1 + rng.integers(0, 2, N)) % 3,
                3 - pilihan - hadiah)
assert np.all(buka != pilihan) and np.all(buka != hadiah)   # aturan permainan terpenuhi
sisa = 3 - pilihan - buka                         # satu-satunya pintu lain yang masih tertutup
g3_tetap = np.mean(pilihan == hadiah)
g3_ganti = np.mean(sisa == hadiah)                # dihitung dari simulasi, bukan ~g3_tetap
print(f"Tetap   : {g3_tetap:.4f}  (teori 0.3333)")
print(f"Berganti: {g3_ganti:.4f}  (teori 0.6667)")
print(f"Tetap + berganti = {g3_tetap + g3_ganti:.4f}")

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (tetap, toleransi 0.01):", cocok(1/3, g3_tetap, 0.01))
print("Cek vs manual (berganti, toleransi 0.01):", cocok(2/3, g3_ganti, 0.01))
```

Output :

```
Tetap   : 0.3386  (teori 0.3333)
Berganti: 0.6614  (teori 0.6667)
Tetap + berganti = 1.0000
Cek vs manual (tetap, toleransi 0.01): SAMA
Cek vs manual (berganti, toleransi 0.01): SAMA
```

Analisis / Kesimpulan :

Dari 100.000 permainan, strategi tetap menang 0,3386 (33,9%) dan strategi berganti menang 0,6614 (66,1%). Keduanya dekat dengan nilai teori 1/3 dan 2/3, dan jumlahnya 1,0000. Jadi sebaiknya pemain

berganti pintu: pilihan awal hanya punya peluang 1/3, dan pintu yang dibuka tidak mengubah itu, sehingga sisa peluang 2/3 ada pada pintu yang tersisa.

BAB III

PERMASALAHAN DISTRIBUSI PROBABILITAS

Tujuh distribusi mengikuti modul 05 (lima diskrit: Hipergeometrik, Binomial, Poisson, Geometrik, Binomial Negatif; dua kontinu: Normal, Eksponensial), ditambah dua soal jembatan ke statistika deskriptif (total 27 soal). Soal berkode berakhiran 0 (HIP0, BIN0, BIN0B, POI0, NOR0, NOR0B) diambil dari contoh pada modul. Notebook: Distribusi_Probabilitas_Python.ipynb.

Distribusi 1 — Binomial (diskrit)

Soal BIN0 : Kegiatan A mendapat dana 3 kali dalam 5 tahun (contoh modul)

Soal: Setiap tahun, selama 5 tahun, dipilih acak satu dari 4 kegiatan (A, B, C, D) untuk didanai. Berapa peluang kegiatan A mendapat dana tepat 3 kali? Buat tabel peluang untuk 0 sampai 5 kali, serta E(X) dan Var(X).

Hitungan manual:

p = 1/4 = 0,25; q = 0,75; n = 5.

f(3; 5, 0,25) = C(5,3) $(0,25)^3$ $(0,75)^2$ = 10 × 0,015625 × 0,5625 = 0,0879

f(0) = 0,2373; f(1) = 0,3955; f(2) = 0,2637; f(3) = 0,0879; f(4) = 0,0146; f(5) = 0,0010 (jumlah = 1)

E(X) = np = 1,25; Var(X) = npq = 0,9375

Source code:

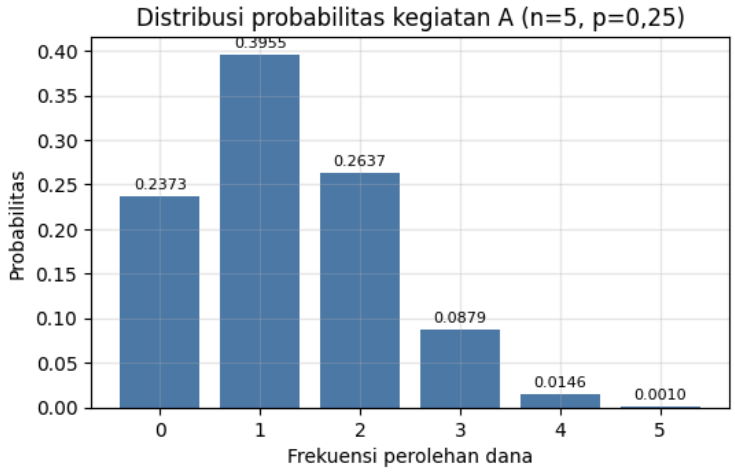
```
n, p = 5, 0.25
x = np.arange(0, n + 1)
pmf = stats.binom.pmf(x, n, p)                          # Excel: BINOM.DIST(x, n, p, FALSE)
tabel = pd.DataFrame({"Jumlah sukses": x, "Jumlah cara": [math.comb(n, i) for i in x],
                      "Probabilitas": pmf.round(4), "Kumulatif F(x)": stats.binom.cdf(x, n, p).round(4)})
print(tabel.to_string(index=False))
bin0_f3, bin0_f0 = pmf[3], pmf[0]
bin0_jumlah = pmf.sum()
bin0_mean, bin0_var = stats.binom.mean(n, p), stats.binom.var(n, p)
print(f"f(3) = {bin0_f3:.4f} | jumlah probabilitas = {bin0_jumlah:.4f} | E(X) = {bin0_mean} | Var(X) = {bin0_var}")

plt.bar(x, pmf, color="#4C78A8")
for xi, pi in zip(x, pmf):
    plt.text(xi, pi + 0.008, f"{pi:.4f}", ha="center", fontsize=8)
plt.xlabel("Frekuensi perolehan dana"); plt.ylabel("Probabilitas"); plt.title("Distribusi probabilitas kegiatan A (n=5, p=0,25)")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (f(3)):", cocok(0.0879, bin0_f3, 0.0001))
print("Cek vs manual (f(0)):", cocok(0.2373, bin0_f0, 0.0001))
print("Cek vs manual (jumlah probabilitas):", cocok(1, bin0_jumlah, 1e-09))
print("Cek vs manual (E(X)):", cocok(1.25, bin0_mean, 1e-09))
print("Cek vs manual (Var(X)):", cocok(0.9375, bin0_var, 1e-09))
```

Output :

```
 Jumlah sukses  Jumlah cara  Probabilitas  Kumulatif F(x)
             0            1        0.2373         0.2373
             1            5        0.3955         0.6328
             2           10        0.2637         0.8965
             3           10        0.0879         0.9844
             4            5        0.0146         0.9990
             5            1        0.0010         1.0000
f(3) = 0.0879 | jumlah probabilitas = 1.0000 | E(X) = 1.25 | Var(X) = 0.9375
```




```
Cek vs manual (f(3)): SAMA
Cek vs manual (f(0)): SAMA
Cek vs manual (jumlah probabilitas): SAMA
Cek vs manual (E(X)): SAMA
Cek vs manual (Var(X)): SAMA
```

Analisis / Kesimpulan :

Peluang kegiatan A mendapatkan pendanaan tepat 3 kali dalam periode 5 tahun berdasarkan pemodelan distribusi binomial diskrit adalah sebesar 0,0879 atau sekitar 8,8%. Hasil analisis distribusi probabilitas menunjukkan bahwa kejadian yang paling mungkin terjadi adalah kegiatan A hanya menerima pendanaan sebanyak 1 kali, dengan peluang tertinggi mencapai 0,3955 atau 39,6%. Skenario di mana kegiatan A sama sekali tidak mendapatkan dana (0 kali) juga memiliki probabilitas yang signifikan, yaitu sebesar 0,2373 atau 23,7%.

Secara keseluruhan, nilai harapan atau rata-rata perolehan dana (E(X)) untuk kegiatan A adalah 1,25 kali, yang secara matematis dihitung dari total 5 tahun dikalikan dengan besaran probabilitas tahunan 0,25. Varians (Var(X)) dari pemodelan distribusi ini tercatat pada angka 0,9375. Akumulasi dari seluruh probabilitas kejadian mulai dari 0 hingga 5 kali perolehan terbukti berjumlah tepat 1, dan pengujian komputasi menggunakan kode Python menghasilkan keluaran yang secara presisi cocok dengan perhitungan manual.

Soal BIN0B : Skewness binomial dan mendekati kurva normal (contoh modul)

Soal: Untuk p = 0,25, hitung koefisien skewness cs = (q − p)/√(npq) untuk n = 5, 10, 20, 50, bandingkan dengan nilai dari `scipy`, dan gambarkan histogram binomial dengan kurva normal N(np, npq). Tafsirkan memakai acuan skewness dari lembar perkuliahan.

Hitungan manual:

n = 5: cs = (0,75 − 0,25)/√(5×0,25×0,75) = 0,5/0,9682 = 0,5164

n = 50: cs = 0,5/√9,375 = 0,1633

Makin besar n, cs mendekati 0 dan bentuk histogram mendekati lonceng (kurva normal).

Catatan: slide modul menyebut q > p sebagai 'negative skew', padahal rumus cs = (q − p)/√(npq) bernilai positif saat q > p, artinya ekor panjang berada di kanan (positive skew). Notebook ini mengikuti tanda cs.

Source code:

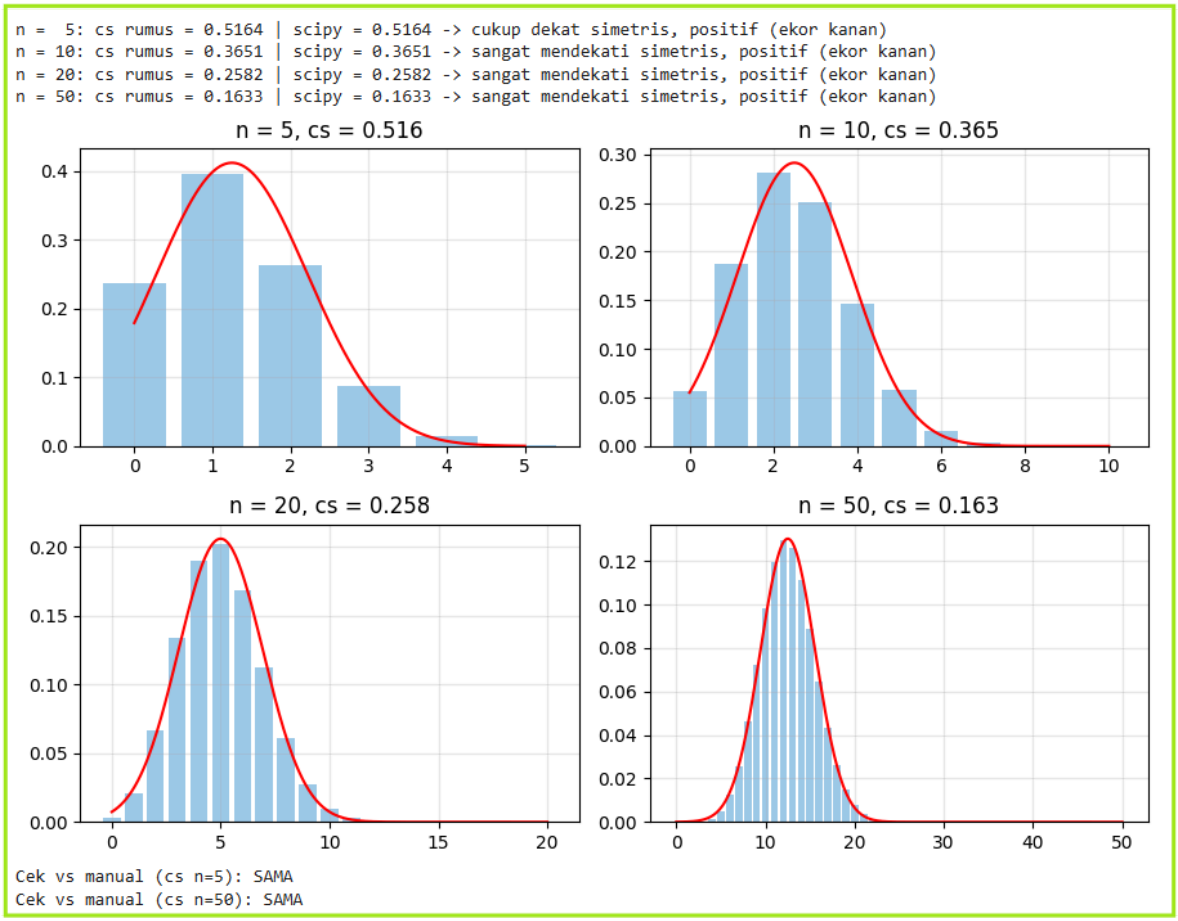
```
p, q = 0.25, 0.75

def tafsir(cs):
    a = abs(cs)
    arah = "positif (ekor kanan)" if cs > 0 else "negatif (ekor kiri)" if cs < 0 else "simetris"
    if a <= 0.5: kat = "sangat mendekati simetris"
    elif a <= 1: kat = "cukup dekat simetris"
    elif a <= 2: kat = "masih dapat ditoleransi"
    else: kat = "kemencengan kuat"
    return f"{kat}, {arah}"

fig, ax = plt.subplots(2, 2, figsize=(9, 6))
for a, n in zip(ax.ravel(), (5, 10, 20, 50)):
    cs_rumus = (q - p) / math.sqrt(n * p * q)
    cs_scipy = float(stats.binom.stats(n, p, moments="s"))
    print(f"n = {n:2d}: cs rumus = {cs_rumus:.4f} | scipy = {cs_scipy:.4f} -> {tafsir(cs_rumus)}")
    xs = np.arange(0, n + 1)
    a.bar(xs, stats.binom.pmf(xs, n, p), color="#9ecae9")
    xx = np.linspace(0, n, 300)
    a.plot(xx, stats.norm.pdf(xx, n * p, math.sqrt(n * p * q)), "r")
    a.set_title(f"n = {n}, cs = {cs_rumus:.3f}")
plt.tight_layout(); plt.show()
bin0b_cs5 = float(stats.binom.stats(5, p, moments="s"))
bin0b_cs50 = float(stats.binom.stats(50, p, moments="s"))

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (cs n=5):", cocok(0.5/math.sqrt(0.9375), bin0b_cs5, 1e-06))
print("Cek vs manual (cs n=50):", cocok(0.5/math.sqrt(9.375), bin0b_cs50, 1e-06))
```

Output / grafik:



Analisis / Kesimpulan :

Berdasarkan analisis perhitungan, koefisien kemencengan (*skewness* / cs) pada saat ukuran sampel n = 5 bernilai 0,5164, yang mengindikasikan kemencengan ke arah kanan (*positive skew*) akibat nilai probabilitas *p* yang tergolong kecil. Namun, seiring dengan peningkatan jumlah sampel hingga mencapai n = 50, nilai *skewness* tersebut terbukti menyusut secara konsisten menjadi 0,1633. Angka ini masuk ke dalam kategori sangat mendekati simetris berdasarkan acuan batas toleransi ±0,5. Secara visual, pemetaan histogram memperlihatkan bahwa semakin besar nilai n, maka bentuk distribusinya akan semakin proporsional menyerupai lonceng, dan garis kurva normal yang direpresentasikan dengan warna merah akan semakin tepat menutupi batang-batang histogram binomial tersebut. Fakta matematis dan visual ini secara meyakinkan membuktikan bahwa pemodelan distribusi binomial dapat didekati dan dipetakan secara akurat menggunakan pendekatan distribusi normal, asalkan nilai parameter n yang digunakan cukup besar.

Soal BIN1 : Tepat 2 paket gagal

Soal: Sebuah pesan dikirim dalam 10 paket. Tiap paket gagal terkirim dengan peluang 0,1 (independen). Berapa peluang tepat 2 paket gagal? Gambarkan distribusi lengkapnya.

Hitungan manual:

X ~ Bin(10; 0,1). $P(X=2) = C(10,2)(0,1)^2(0,9)^8 = 45 \times 0{,}01 \times 0{,}43047 = 0{,}19371$

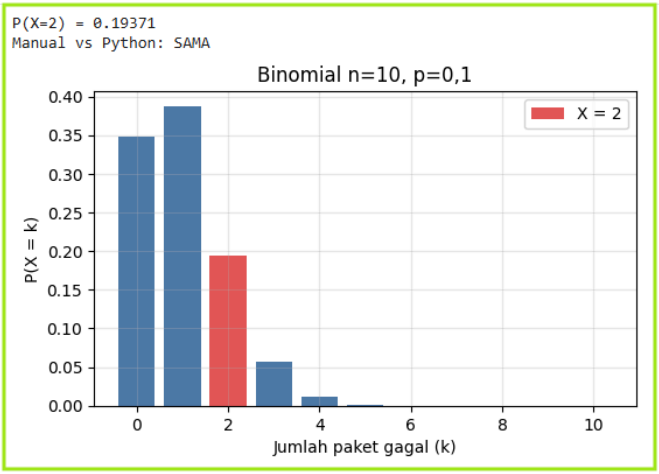
Source code:

```
n, p = 10, 0.1
bin1_p = stats.binom.pmf(2, n, p)
print(f"P(X=2) = {bin1_p:.5f}")
print("Manual vs Python:", cocok(0.19371, bin1_p, 1e-5))

k = np.arange(0, n + 1)
plt.bar(k, stats.binom.pmf(k, n, p), color="#4C78A8")
plt.bar(2, bin1_p, color="#E45756", label="X = 2")
plt.xlabel("Jumlah paket gagal (k)"); plt.ylabel("P(X = k)"); plt.title("Binomial n=10, p=0,1"); plt.legend()
plt.show()
```

Output / grafik:

```
P(X=2) = 0.19371
Manual vs Python: SAMA
```



Analisis / Kesimpulan :

Berdasarkan perhitungan distribusi binomial, peluang terjadinya kegagalan pengiriman pada tepat 2 paket data dari total 10 paket adalah sebesar 0,19371 atau sekitar 19,4%. Secara visual, grafik histogram memperlihatkan bentuk distribusi yang condong ke arah kanan (*positive skew*), yang mengindikasikan bahwa jumlah kegagalan yang paling mungkin terjadi berada pada angka 0 atau 1 paket. Meskipun demikian, peluang terjadinya 2 buah kegagalan paket, yang ditandai dengan batang merah pada grafik, masih dikategorikan cukup umum dan sangat mungkin terjadi. Sebaliknya, kemungkinan terjadinya kegagalan pengiriman yang melibatkan lebih dari 4 buah paket memiliki probabilitas yang sangat mendekati nol atau hampir tidak pernah terjadi sama sekali dalam skenario distribusi ini. Hasil komputasi dari kode Python untuk nilai $P(X = 2)$ juga terbukti sama persis dengan perhitungan secara manual.

Soal BIN2 : Paling banyak 1 paket gagal (cdf)

Soal: Dari soal BIN1, berapa peluang paling banyak 1 paket gagal?

Hitungan manual:

$$P(X \leq 1) = P(0) + P(1) = 0{,}9^{10} + 10(0{,}1)(0{,}9)^{9} = 0{,}34868 + 0{,}38742 = 0{,}73610$$

Source code:

```
bin2_cdf = stats.binom.cdf(1, 10, 0.1)
bin2_jumlah = stats.binom.pmf(0, 10, 0.1) + stats.binom.pmf(1, 10, 0.1)
print(f"P(X <= 1) via cdf   = {bin2_cdf:.5f}")
print(f"P(0) + P(1)         = {bin2_jumlah:.5f}")
print("Manual vs Python:", cocok(0.73610, bin2_cdf, 1e-5))
```

Output / grafik:

```
P(X <= 1) via cdf   = 0.73610
P(0) + P(1)         = 0.73610
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas kumulatif, peluang terjadinya kegagalan pengiriman pada paling banyak 1 paket adalah sebesar 0,73610 atau sekitar 73,6%. Nilai probabilitas kumulatif ini secara matematis diperoleh dari penjumlahan antara kejadian tidak ada paket yang gagal (P(0) sebesar 0,34868) dan kejadian tepat 1 paket yang gagal (P(1) sebesar 0,38742).

Penggunaan fungsi cdf (*Cumulative Distribution Function*) pada pustaka Python terbukti menghasilkan nilai yang sama persis dengan metode penjumlahan pmf (*Probability Mass Function*) secara manual. Sebagai implikasi logis dari hasil probabilitas kumulatif tersebut, dapat ditarik kesimpulan dari komplemennya, yakni masih terdapat probabilitas sekitar 26,4% bahwa sebuah pesan akan mengalami tingkat kegagalan yang lebih parah, yaitu pada 2 paket atau lebih. Seluruh hasil eksekusi komputasi juga tervalidasi akurat dan cocok secara mutlak dengan perhitungan manual.

Distribusi 2 — Poisson (diskrit)

Soal POI0 : Salah cetak printer (contoh modul)

Soal: Sebuah printer membuat kesalahan acak rata-rata 2 kesalahan per halaman. Hitung peluang tepat satu salah cetak dalam satu halaman, peluang paling banyak 2 kesalahan, serta koefisien skewness.

Hitungan manual:

$\lambda = 2$, x = 1: $f(1; 2) = 2^1 e^{-2} / 1! = 2/e^2 = 0{,}2707$

$F(2) = e^{-2}(1 + 2 + 2) = 5e^{-2} = 0{,}6767$

Skewness cs = $\lambda^{(-1/2)} = 1/\sqrt{2} = 0{,}7071$

Source code:

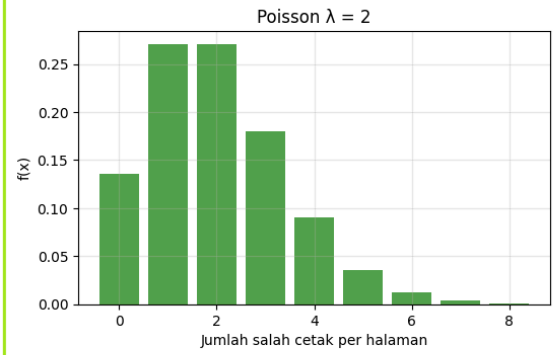
```
lam = 2
poi0_p1 = stats.poisson.pmf(1, lam)              # Excel: POISSON.DIST(1, 2, FALSE)
poi0_cdf2 = stats.poisson.cdf(2, lam)
poi0_skew = float(stats.poisson.stats(lam, moments="s"))
print(f"f(1; 2) = {poi0_p1:.4f} | F(2) = {poi0_cdf2:.4f} | skewness = {poi0_skew:.4f} (rumus 1/sqrt(lambda) = {1/math.sqrt(lam):.4f})")
k = np.arange(0, 9)
print(pd.DataFrame({"x": k, "f(x)": stats.poisson.pmf(k, lam).round(4), "F(x)": stats.poisson.cdf(k, lam).round(4)}).to_string(index=False))
plt.bar(k, stats.poisson.pmf(k, lam), color="#54A24B")
plt.xlabel("Jumlah salah cetak per halaman"); plt.ylabel("f(x)"); plt.title("Poisson λ = 2")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (f(1;2)):", cocok(0.2707, poi0_p1, 0.0001))
print("Cek vs manual (F(2)):", cocok(5*math.exp(-2), poi0_cdf2, 1e-09))
print("Cek vs manual (skewness):", cocok(1/math.sqrt(2), poi0_skew, 1e-09))
```

Output :

```
f(1; 2) = 0.2707 | F(2) = 0.6767 | skewness = 0.7071 (rumus 1/sqrt(lambda) = 0.7071)
 x   f(x)   F(x)
 0 0.1353 0.1353
 1 0.2707 0.4060
 2 0.2707 0.6767
 3 0.1804 0.8571
 4 0.0902 0.9473
 5 0.0361 0.9834
 6 0.0120 0.9955
 7 0.0034 0.9989
 8 0.0009 0.9998
Cek vs manual (f(1;2)): SAMA
Cek vs manual (F(2)): SAMA
Cek vs manual (skewness): SAMA
```

```
f(1; 2) = 0.2707 | F(2) = 0.6767 | skewness = 0.7071 (rumus 1/sqrt(lambda) = 0.7071)
 x   f(x)   F(x)
 0 0.1353 0.1353
 1 0.2707 0.4060
 2 0.2707 0.6767
 3 0.1804 0.8571
 4 0.0902 0.9473
 5 0.0361 0.9834
 6 0.0120 0.9955
 7 0.0034 0.9989
 8 0.0009 0.9998
```




```
Cek vs manual (f(1;2)): SAMA
Cek vs manual (F(2)): SAMA
Cek vs manual (skewness): SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan distribusi Poisson dengan nilai rata-rata kejadian (λ) sebesar 2, peluang untuk mendapati tepat 1 kesalahan cetak pada suatu halaman adalah sebesar 0,2707 atau sekitar 27,1%. Nilai peluang ini terbukti setara dan valid sesuai dengan data pada modul. Selanjutnya, melalui perhitungan fungsi probabilitas kumulatif (CDF), probabilitas untuk menjumpai kejadian maksimal atau paling banyak 2 kesalahan cetak dalam satu halaman adalah sebesar 0,6767 atau sekitar 67,7%.

Adapun untuk nilai koefisien *skewness* (kemencengan distribusi) pada model Poisson ini tercatat pada angka 0,7071 yang bernilai positif. Hal ini menandakan bahwa bentuk distribusi grafiknya akan cenderung miring ke arah kanan (*positive skew*) yang disebabkan oleh kecilnya angka parameter λ. Meskipun demikian, merujuk pada standar acuan dalam lembar perkuliahan, nilai *skewness* 0,7071 tersebut masih berada di dalam batas interval toleransi ±1, yang mengindikasikan bahwa distribusi ini secara statistik masih dapat dikategorikan cukup dekat dengan bentuk yang simetris. Seluruh pengujian komputasi dengan Python sukses memvalidasi hasil yang sepenuhnya sama dengan proses perhitungan analitik manual.

Soal POI1 : Tepat 6 request per menit

Soal: Sebuah server menerima rata-rata 4 request per menit (Poisson). Berapa peluang tepat 6 request dalam satu menit?

Hitungan manual:

$\lambda = 4$. $P(X=6) = e^{-4}\, 4^6 / 6! = 0{,}018316 \times 4096 / 720 = 0{,}10420$

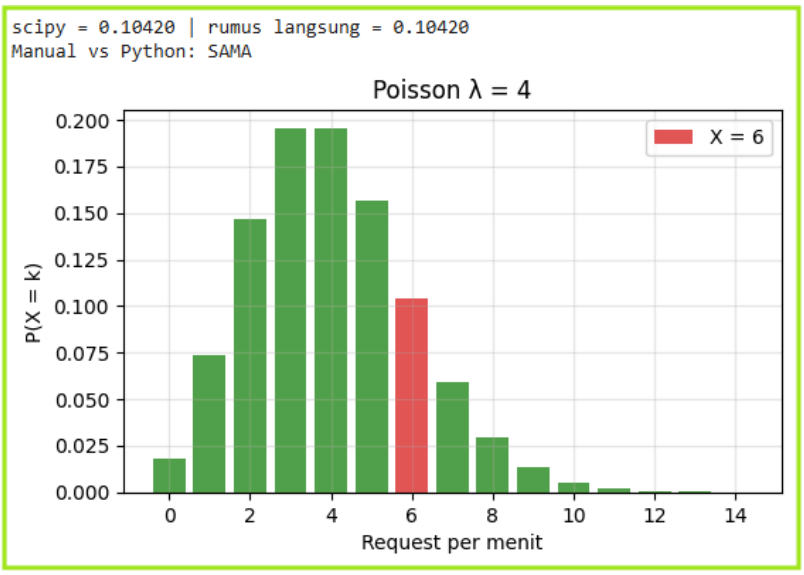
Source code:

```python
lam = 4
poi1_p = stats.poisson.pmf(6, lam)
poi1_rumus = math.exp(-lam) * lam**6 / math.factorial(6)
print(f"scipy = {poi1_p:.5f} | rumus langsung = {poi1_rumus:.5f}")
print("Manual vs Python:", cocok(0.10420, poi1_p, 1e-5))

k = np.arange(0, 15)
plt.bar(k, stats.poisson.pmf(k, lam), color="#54A24B")
plt.bar(6, poi1_p, color="#E45756", label="X = 6")
plt.xlabel("Request per menit"); plt.ylabel("P(X = k)"); plt.legend(); plt.title("Poisson λ = 4")
plt.show()
```

Output :

```
scipy = 0.10420 | rumus langsung = 0.10420
Manual vs Python: SAMA
```



Analisis / Kesimpulan :

Dengan rata-rata λ = 4 request per menit, peluang server menerima tepat 6 request dalam satu menit adalah 0,1042 (10,4%). Pada grafik, batang tertinggi ada di 3 dan 4 request (dekat rata-rata), lalu peluangnya mengecil ke kiri dan kanan. Jadi 6 request masih mungkin terjadi tetapi lebih jarang daripada 3 atau 4 request. Hasil Python sama dengan hitungan manual.

Soal POI2 : Paling banyak 2 request

Soal: Pada server yang sama (λ = 4 per menit), berapa peluang paling banyak 2 request dalam satu menit?

Hitungan manual:

$P(X \leq 2) = e^{-4}\ (1 + 4 + 8) = 13\ e^{-4} = 0{,}2381$

Source code:

```
poi2_p = stats.poisson.cdf(2, 4)
print(f"P(X <= 2) = {poi2_p:.5f}")
print("Manual vs Python:", cocok(13 * math.exp(-4), poi2_p, 1e-8))
```

Output :

```
P(X <= 2) = 0.23810
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas kumulatif menggunakan distribusi Poisson dengan rata-rata beban (λ) sebesar 4 *request* per menit, peluang sebuah server menerima paling banyak 2 *request* dalam kurun waktu satu menit adalah sebesar 0,2381 atau sekitar 23,8%. Secara praktis operasional, nilai probabilitas ini mengindikasikan bahwa rata-rata pada 1 dari setiap 4 menit, kondisi server tersebut akan cenderung relatif sepi dari lonjakan permintaan. Selain itu, eksekusi komputasi menggunakan fungsi cdf (*Cumulative Distribution Function*) pada pustaka statistik Python terbukti menghasilkan nilai yang sepenuhnya identik dan akurat jika disandingkan dengan penjabaran rumus matematis secara manual, yakni $13e^{-4}$.

Soal POI3 : Minimal 1 request dalam 30 detik (penskalaan λ)

Soal: Dengan laju 4 request per menit, berapa peluang minimal 1 request datang dalam 30 detik?

Hitungan manual:

Dalam 30 detik $\lambda = 4 \times 0{,}5 = 2$. $P(X \geq 1) = 1 - P(0) = 1 - e^{-2} = 0{,}8647$

Source code:

```
lam30 = 4 * 0.5
poi3_p = stats.poisson.sf(0, lam30)       # P(X >= 1)
print(f"lambda 30 detik = {lam30}")
print(f"P(X >= 1) = {poi3_p:.5f}")
print("Manual vs Python:", cocok(1 - math.exp(-2), poi3_p, 1e-8))
```

Output :

```
lambda 30 detik = 2.0
P(X >= 1) = 0.86466
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan distribusi Poisson, parameter rata-rata kedatangan (λ) mutlak harus disesuaikan terlebih dahulu dengan proporsi panjang interval waktu yang sedang ditinjau. Karena rentang waktu observasi dipersempit menjadi 30 detik (setengah menit) dari laju awal 4 *request* per menit, maka nilai λ harus diskalakan menjadi 2. Melalui penyesuaian parameter tersebut dan penggunaan aturan komplemen probabilitas, peluang server untuk menerima minimal satu buah *request* tercatat sebesar 0,8647 atau sekitar 86,5%. Secara praktis operasional, tingginya angka probabilitas ini mengindikasikan bahwa server tersebut hampir dipastikan selalu menerima setidaknya satu buah *request* pada setiap interval setengah menit. Pengujian komputasi menggunakan kode Python juga terbukti berhasil memvalidasi penskalaan waktu tersebut dengan hasil keluaran yang akurat dan sama persis dengan perhitungan secara manual.

Soal POI6 : Pendekatan Poisson untuk Binomial

Soal: Dari 1.000 transaksi, tiap transaksi bermasalah dengan peluang 0,003. Hitung prob(tepat 2 bermasalah) dengan distribusi Binomial, lalu bandingkan dengan pendekatan Poisson (λ = np = 3).

Hitungan manual:

Binomial: $C(1000,2)(0{,}003)^2(0{,}997)^{998} \approx 0{,}22415$

Poisson: $e^{-3} \cdot 3^2 / 2! = 0{,}22404$

Source code:

```
In [17]:  n, p = 1000, 0.003
          poi6_b = stats.binom.pmf(2, n, p)
          poi6_p = stats.poisson.pmf(2, n * p)
          print(f"Binomial = {poi6_b:.5f}")
          print(f"Poisson  = {poi6_p:.5f}")
          print(f"Galat relatif = {abs(poi6_b - poi6_p) / poi6_b:.4%}")

          # Pembanding dengan angka pada hitungan manual
          print("Cek vs manual (binomial):", cocok(0.22415, poi6_b, 1e-05))
          print("Cek vs manual (Poisson):", cocok(0.22404, poi6_p, 1e-05))
```

Output :

```
Out [17]:  Binomial = 0.22415
           Poisson  = 0.22404
           Galat relatif = 0.0499%
           Cek vs manual (binomial): SAMA
           Cek vs manual (Poisson): SAMA
```

Analisis / Kesimpulan :

Peluang tepat 2 transaksi bermasalah adalah 0,22415 menurut Binomial dan 0,22404 menurut Poisson, selisihnya hanya sekitar 0,05%. Jadi kalau n besar (1.000) dan p kecil (0,003), Poisson dengan $\lambda = np = 3$ bisa menggantikan Binomial dan hitungannya lebih sederhana. Hasil Python sama dengan hitungan manual.

Soal POI7 : Menentukan kapasitas server

Soal: Rata-rata 5 request/menit. Server memproses maksimal 8 request/menit. (a) Berapa peluang server kelebihan beban (> 8)? (b) Berapa kapasitas minimum per menit agar peluang kelebihan beban < 1%?

Hitungan manual:

(a) $P(X > 8) = 1 - P(X \leq 8) \approx 0{,}0681$

(b) Cari k terkecil dengan $P(X > k) < 0{,}01$ → $P(X > 10) = 0{,}0137$ (masih lebih besar), $P(X > 11) = 0{,}0055$ → $k = 11$.

Source code:

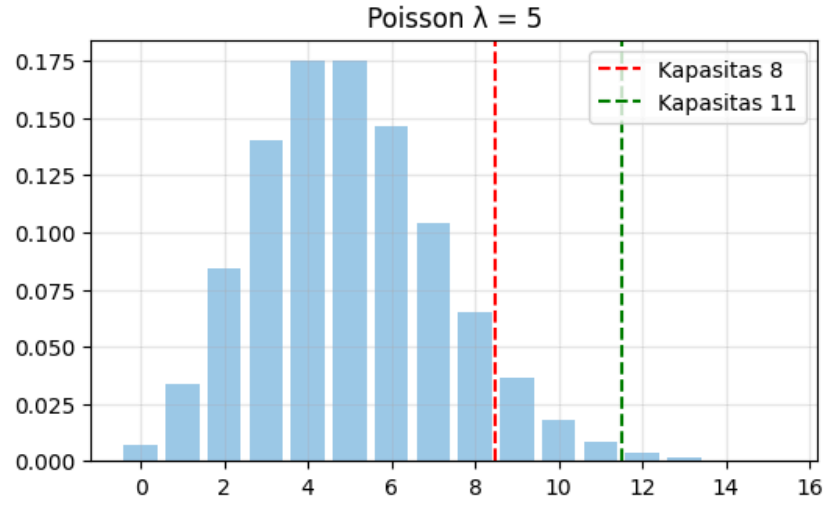
```
lam = 5
poi7_a = stats.poisson.sf(8, lam)
k = 0
while stats.poisson.sf(k, lam) >= 0.01:
    k += 1
poi7_k = k
print(f"(a) P(X > 8) = {poi7_a:.4f}")
print(f"(b) Kapasitas minimum = {poi7_k} request/menit (P overload = {stats.poisson.sf(poi7_k, lam):.4f})")
print("Cek dengan ppf:", int(stats.poisson.ppf(0.99, lam)))

kk = np.arange(0, 16)
plt.bar(kk, stats.poisson.pmf(kk, lam), color="#9ecae9")
plt.axvline(8.5, color="red", ls="--", label="Kapasitas 8")
plt.axvline(poi7_k + 0.5, color="green", ls="--", label=f"Kapasitas {poi7_k}")
plt.legend(); plt.title("Poisson λ = 5"); plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (P(X>8)):", cocok(0.0681, poi7_a, 0.0001))
print("Cek vs manual (kapasitas, toleransi 0.5):", cocok(11, poi7_k, 0.5))
```

Output :

```
(a) P(X > 8) = 0.0681
(b) Kapasitas minimum = 11 request/menit (P overload = 0.0055)
Cek dengan ppf: 11
```




```
Cek vs manual (P(X>8)): SAMA
Cek vs manual (kapasitas, toleransi 0.5): SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas menggunakan distribusi Poisson dengan rata-rata beban 5 *request* per menit, peluang sebuah server dengan kapasitas proses 8 *request* per menit mengalami kelebihan beban atau *overload* (X > 8) adalah sebesar 0,0681 atau 6,81%. Angka ini diperoleh melalui perhitungan komplemen probabilitas kumulatif, yakni mengurangkan probabilitas total dengan peluang server memproses 8 *request* atau kurang.

Untuk menekan risiko *overload* hingga berada di bawah ambang batas 1%, pengujian matematis menunjukkan bahwa peningkatan kapasitas menjadi 10 *request* per menit masih menyisakan risiko kegagalan sebesar 0,0137 atau 1,37%. Oleh karena itu, kapasitas minimum wajib disesuaikan menjadi 11 *request* per menit, yang terbukti secara komputasi mampu menjatuhkan probabilitas *overload* hingga ke level 0,0055 atau 0,55%.

Distribusi 3 — Geometrik (diskrit)

Soal GEO1 : Login berhasil pada percobaan ke-3

Soal: Seorang penyerang menebak password, tiap tebakan benar dengan peluang 0,2. Berapa peluang tebakan benar pertama terjadi pada percobaan ke-3?

Hitungan manual:

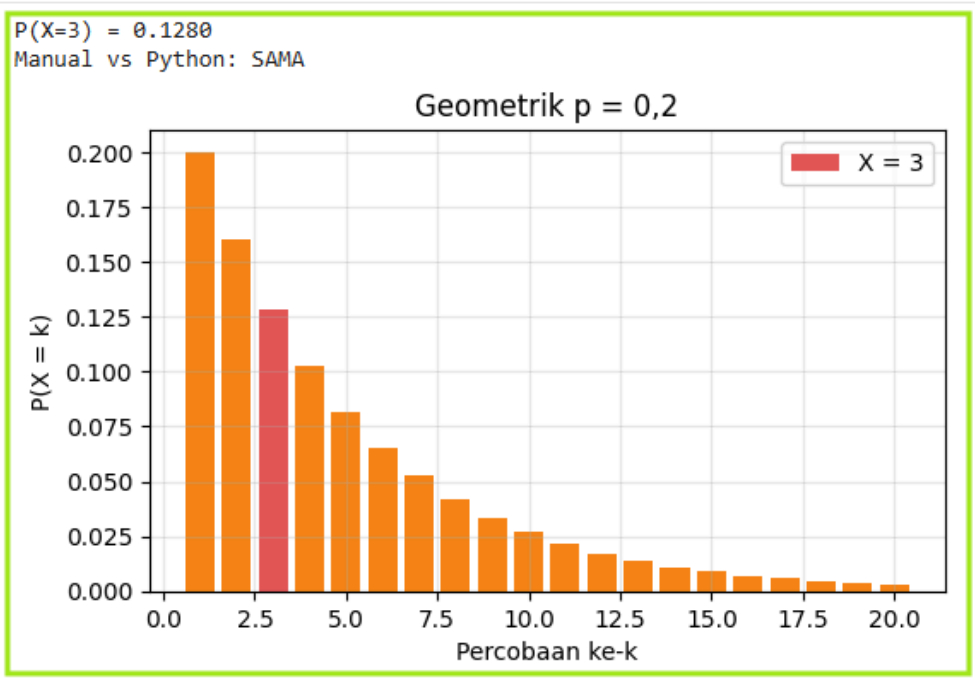
$P(X=3) = (0,8)^2 \times 0,2 = 0,64 \times 0,2 = 0,128$

Source code:

```
p = 0.2
geo1_p = stats.geom.pmf(3, p)
print(f"P(X=3) = {geo1_p:.4f}")
print("Manual vs Python:", cocok(0.128, geo1_p, 1e-9))

k = np.arange(1, 21)
plt.bar(k, stats.geom.pmf(k, p), color="#F58518")
plt.bar(3, geo1_p, color="#E45756", label="X = 3")
plt.xlabel("Percobaan ke-k"); plt.ylabel("P(X = k)"); plt.legend(); plt.title("Geometrik p = 0,2")
plt.show()
```

Output :



Analisis / Kesimpulan :

Berdasarkan perhitungan model probabilitas geometrik, peluang bahwa sebuah tebakan kata sandi (*password*) baru akan berhasil tepat pada percobaan yang ke-3 adalah sebesar 0,128 atau 12,8%. Pemetaan visual pada grafik menunjukkan pola distribusi yang menurun secara monoton. Pola ini menegaskan karakteristik utama distribusi geometrik, di mana probabilitas keberhasilan tertinggi selalu berada tepat pada percobaan yang pertama (sebesar p = 0,2). Untuk setiap percobaan berikutnya yang berurutan, nilai probabilitas akan secara konsisten mengalami penyusutan berdasarkan deret ukur atau deret geometris. Seluruh pengujian secara komputasi menggunakan Python terbukti akurat dan menghasilkan keluaran probabilitas yang sama persis dengan perhitungan manual.

Soal GEO2 : Berhasil dalam 5 percobaan

Soal: Dengan p = 0,2, berapa peluang berhasil paling lambat pada percobaan ke-5?

Hitungan manual:

$P(X \leq 5) = 1 - (0{,}8)^5 = 1 - 0{,}32768 = 0{,}67232$

Source code:

```
geo2_p = stats.geom.cdf(5, 0.2)
print(f"P(X <= 5) = {geo2_p:.5f}")
print("Manual vs Python:", cocok(1 - 0.8**5, geo2_p, 1e-9))
```

Output :

```
P(X <= 5) = 0.67232
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas kumulatif menggunakan distribusi geometrik dengan tingkat keberhasilan (p) sebesar 0,2, peluang untuk mencapai kesuksesan (berhasil menebak kata sandi) paling lambat pada percobaan ke-5 adalah sebesar 0,67232 atau sekitar 67,2%. Perhitungan matematis ini menunjukkan sebuah prinsip keamanan siber yang vital: meskipun probabilitas kesuksesan pada setiap percobaan tunggal tergolong kecil, nilai akumulasi probabilitas kumulatifnya akan membengkak menjadi sangat besar seiring dengan bertambahnya jumlah kesempatan yang diberikan. Fakta matematis inilah yang menjadi dasar rasionalisasi dan keharusan mengapa setiap sistem autentikasi keamanan mutlak perlu membatasi atau mengunci jumlah maksimal percobaan *login* dari penggunanya. Seluruh pengujian komputasional yang dieksekusi menggunakan fungsi `cdf` pada pustaka Python terbukti sukses memvalidasi tingkat akurasi yang sepenuhnya identik dengan penjabaran manualnya

Soal GEO7 : Simulasi Geometrik

Soal: Bangkitkan 100.000 sampel X ~ Geom(0,25), bandingkan mean dan varians dengan teori, dan gambarkan histogram terhadap pmf.

Hitungan manual:

E[X] = 1/0,25 = 4; Var = 0,75/0,0625 = 12

Source code:

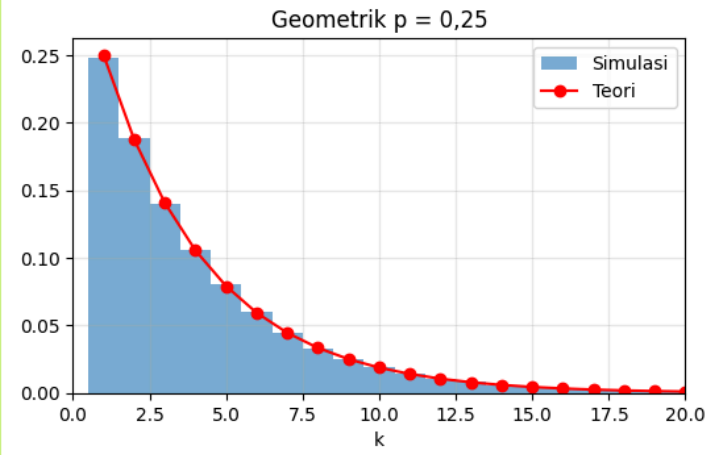
```
p = 0.25
data = stats.geom.rvs(p, size=100_000, random_state=5)
geo7_mean, geo7_var = data.mean(), data.var()
print(f"Simulasi: mean = {geo7_mean:.3f}, var = {geo7_var:.3f}")
print(f"Teori   : mean = {1/p}, var = {(1-p)/p**2}")

k = np.arange(1, 21)
plt.hist(data, bins=np.arange(0.5, 21.5), density=True, alpha=0.6, label="Simulasi")
plt.plot(k, stats.geom.pmf(k, p), "ro-", label="Teori")
plt.xlim(0, 20); plt.legend(); plt.xlabel("k"); plt.title("Geometrik p = 0,25")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (mean, toleransi 0.1):", cocok(4, geo7_mean, 0.1))
print("Cek vs manual (var, toleransi 0.5):", cocok(12, geo7_var, 0.5))
```

Output :

```
Simulasi: mean = 4.010, var = 12.023
Teori   : mean = 4.0, var = 12.0
```




```
Cek vs manual (mean, toleransi 0.1): SAMA
Cek vs manual (var, toleransi 0.5): SAMA
```

Analisis / Kesimpulan :

Berdasarkan eksekusi simulasi komputasi yang membangkitkan 100.000 sampel acak pada model distribusi geometrik dengan tingkat probabilitas sukses (p) sebesar 0,25, diperoleh nilai rata-rata (*mean*) empiris sebesar 4,010 beserta nilai varians sebesar 12,023. Hasil taksiran dari simulasi ini terbukti sangat presisi dan mendekati angka perhitungan teoritis matematisnya, di mana nilai *mean* ekspektasi adalah 4 dan variansnya adalah 12.

Secara visual, grafik histogram yang merepresentasikan data simulasi tersebut terlihat sangat selaras dan mengikuti alur kurva *Probability Mass Function* (PMF) teoritisnya yang terus menurun secara monoton. Pemetaan pola grafik ini secara empiris menegaskan karakteristik fundamental yang melekat pada distribusi geometrik, yakni memiliki bentuk distribusi yang condong ke arah kanan (*positive*

skew) serta diiringi dengan rentangan ekor yang panjang (*long tail*). Seluruh algoritma pengujian pada skrip Python juga berhasil memvalidasi kesamaan hasil antara simulasi komputasional dengan perhitungan manual dalam batas toleransi yang ditetapkan.

Distribusi 4 — Hipergeometrik (diskrit)

Soal HIP0 : Stasiun penakar hujan rusak (contoh modul)

Soal: Suatu DAS memiliki N = 12 stasiun penakar hujan dan k = 2 di antaranya rusak. Dipilih n = 6 stasiun secara acak. Berapa peluang terpilih stasiun rusak sejumlah x = 2, 1, atau 0? Hitung juga E(X) dan Var(X).

Hitungan manual:

f(x; N, n, k) = C(k,x) C(N−k, n−x) / C(N,n), dengan C(12,6) = 924.

f(2) = C(2,2) C(10,4)/924 = 210/924 = 0,2273

f(1) = C(2,1) C(10,5)/924 = 504/924 = 0,5454

f(0) = C(2,0) C(10,6)/924 = 210/924 = 0,2273

E(X) = nk/N = 6×2/12 = 1

Var(X) = n k (N−k)(N−n) / (N² (N−1)) = 6×2×10×6 / (144×11) = 720/1584 = 0,4545

Source code:

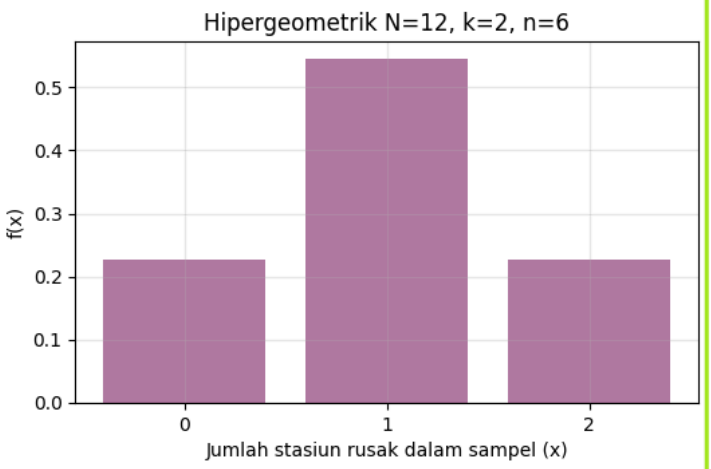
```python
N, k, n = 12, 2, 6                      # notasi modul: populasi N, sukses k, sampel n
x = np.arange(0, 3)
pmf = stats.hypergeom.pmf(x, N, k, n)    # Excel: HYPGEOM.DIST(x, n, k, N, FALSE)  (urutan argumen berbeda!)
cdf = stats.hypergeom.cdf(x, N, k, n)    # Excel: HYPGEOM.DIST(x, n, k, N, TRUE)
tabel = pd.DataFrame({"x": x, "f(x)": pmf, "F(x)": cdf,
                      "rumus kombinasi": [math.comb(k, i) * math.comb(N - k, n - i) / math.comb(N, n) for i in x]})
print(tabel.round(4).to_string(index=False))
hip0_f2, hip0_f1, hip0_f0 = pmf[2], pmf[1], pmf[0]
hip0_e = (x * pmf).sum()                                  # E(X) = jumlah x f(x)
hip0_e_rumus = n * k / N
hip0_var = stats.hypergeom.var(N, k, n)
hip0_var_rumus = n * k * (N - k) * (N - n) / (N**2 * (N - 1))
print(f"E(X) = {hip0_e:.4f} (rumus nk/N = {hip0_e_rumus:.4f}) | Var(X) = {hip0_var:.4f} (rumus = {hip0_var_rumus:.4f})")

plt.bar(x, pmf, color="#B279A2")
plt.xticks(x); plt.xlabel("Jumlah stasiun rusak dalam sampel (x)"); plt.ylabel("f(x)")
plt.title("Hipergeometrik N=12, k=2, n=6")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (f(2)):", cocok(0.2273, hip0_f2, 0.0001))
print("Cek vs manual (f(1)):", cocok(0.5454, hip0_f1, 0.0001))
print("Cek vs manual (f(0)):", cocok(0.2273, hip0_f0, 0.0001))
print("Cek vs manual (E(X)):", cocok(1, hip0_e, 1e-09))
print("Cek vs manual (Var(X)):", cocok(720/1584, hip0_var, 1e-09))
```

Output :

```
 x   f(x)   F(x)  rumus kombinasi
 0 0.2273 0.2273          0.2273
 1 0.5455 0.7727          0.5455
 2 0.2273 1.0000          0.2273
E(X) = 1.0000 (rumus nk/N = 1.0000) | Var(X) = 0.4545 (rumus = 0.4545)
```




```
Cek vs manual (f(2)): SAMA
Cek vs manual (f(1)): SAMA
Cek vs manual (f(0)): SAMA
Cek vs manual (E(X)): SAMA
Cek vs manual (Var(X)): SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas menggunakan model distribusi hipergeometrik, peluang untuk mendapati tepat 0, 1, dan 2 stasiun penakar hujan yang rusak dalam sebuah sampel acak secara berurutan adalah sebesar 0,2273, 0,5455, dan 0,2273. Skenario kejadian yang paling mungkin untuk terjadi adalah terpilihnya tepat 1 stasiun yang rusak di dalam sampel, dengan nilai probabilitas dominan mencapai 54,5%. Akumulasi matematis dari ketiga probabilitas tersebut bernilai tepat 1, di mana terdapat sedikit perbedaan pembulatan desimal dibandingkan modul (yang menuliskan 0,5454) karena nilai presisinya adalah 0,54545.

Lebih lanjut, nilai ekspektasi E(X) atau rata-rata jumlah stasiun rusak yang diharapkan dari penarikan sampel ini adalah 1 stasiun, sebuah nilai yang terbukti ekuivalen dengan rumus teoritis nk/N. Nilai varians Var(X) tercatat pada angka 0,4545. Terdapat koreksi penting terkait rumus varians yang tercantum pada slide modul awal; rumus yang diketik (N–n)(N–n) adalah keliru, dan perhitungan dalam analisis komputasional ini telah menggunakan bentuk koreksi matematis yang benar, yakni $(N - k)(N - n)$. Seluruh pengujian menggunakan skrip Python berhasil memvalidasi kesamaan hasil secara mutlak dengan perhitungan matematis secara manual.

Soal HIP1 : Tepat 1 modul berbug dalam sampel

Soal: Dari 50 modul, 8 modul mengandung bug. Diambil 5 modul acak untuk diuji. Berapa peluang tepat 1 modul berbug?

Hitungan manual:

P(X=1) = C(8,1) C(42,4) / C(50,5) = 8 × 111.930 / 2.118.760 = 0,42262

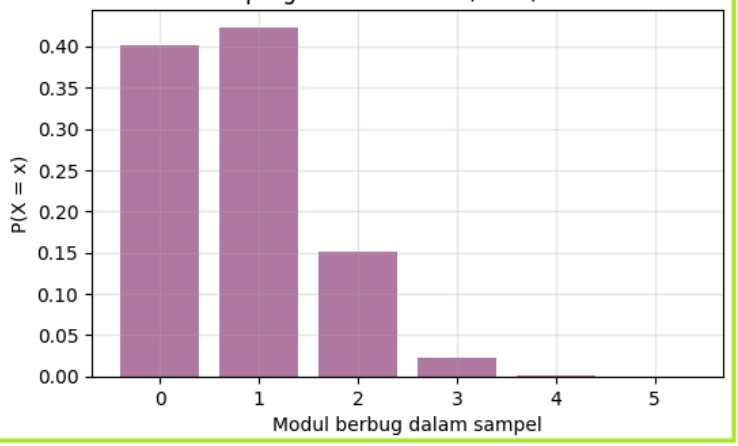
Source code:

```
N, k, n = 50, 8, 5            # populasi N, sukses k, sampel n (notasi modul)
hip1_p = stats.hypergeom.pmf(1, N, k, n)
hip1_cek = math.comb(8, 1) * math.comb(42, 4) / math.comb(50, 5)
print(f"scipy = {hip1_p:.5f} | rumus kombinasi = {hip1_cek:.5f}")
print("Manual vs Python:", cocok(0.42262, hip1_p, 1e-5))

x = np.arange(0, 6)
plt.bar(x, stats.hypergeom.pmf(x, N, k, n), color="#B279A2")
plt.xlabel("Modul berbug dalam sampel"); plt.ylabel("P(X = x)"); plt.title("Hipergeometrik N=50, k=8, n=5")
plt.show()
```

Output :

```
scipy = 0.42262 | rumus kombinasi = 0.42262
Manual vs Python: SAMA
```



Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas menggunakan distribusi hipergeometrik dari total populasi 50 modul yang diuji, peluang untuk menemukan tepat 1 buah modul yang mengandung *bug* di dalam penarikan acak 5 modul sampel adalah sebesar 0,42262 atau sekitar 42,3%. Pengamatan secara visual terhadap pemetaan grafik distribusinya menunjukkan bahwa kejadian penemuan 1 modul berbug ini merupakan kejadian dengan nilai probabilitas yang paling tinggi atau paling mungkin terjadi di antara kemungkinan lainnya. Seluruh pengujian komputasi menggunakan fungsi bawaan dari pustaka `scipy` pada Python terbukti menghasilkan angka yang identik dan akurat secara mutlak dengan penjabaran rumus kombinasi matematika secara manual.

Soal HIP2 : Sampel lolos tanpa mendeteksi bug

Soal: Pada kasus HIP1, berapa peluang sampel tidak berisi modul berbug sama sekali (bug tak terdeteksi), dan berapa peluang minimal satu bug terdeteksi?

Hitungan manual:

P(X=0) = C(42,5)/C(50,5) = 850.668 / 2.118.760 = 0,40149

$P(X \geq 1) = 1 - 0{,}40149 = 0{,}59851$

Source code:

```
hip2_0 = stats.hypergeom.pmf(0, 50, 8, 5)
hip2_min1 = stats.hypergeom.sf(0, 50, 8, 5)
print(f"P(X=0)  = {hip2_0:.5f}")
print(f"P(X>=1) = {hip2_min1:.5f}")
print("Manual vs Python:", cocok(0.40149, hip2_0, 1e-5), cocok(0.59851, hip2_min1, 1e-5))
```

Output :

```
P(X=0)  = 0.40149
P(X>=1) = 0.59851
Manual vs Python: SAMA SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan distribusi hipergeometrik, peluang bahwa pengujian terhadap 5 modul sampel secara acak sama sekali luput atau tidak mendeteksi adanya *bug* (P(X=0)) adalah sebesar 0,40149 atau sekitar 40,1%. Angka probabilitas lolosnya *bug* ini tergolong cukup tinggi, mengingat pada kenyataannya terdapat 16% modul di dalam keseluruhan populasi (8 dari total 50 modul) yang terinfeksi *bug*. Melalui perhitungan komplemen, probabilitas pengujian untuk berhasil mendeteksi setidaknya satu buah *bug* (P(X≥1)) hanya berada di angka 0,59851 atau sekitar 59,9%. Dari perspektif penjaminan kualitas, tingkat probabilitas deteksi sebesar 59,9% ini belum cukup memadai, sehingga besaran ukuran sampel uji secara mutlak perlu diperbesar apabila sistem ingin menghasilkan deteksi yang benar-benar andal. Seluruh eksekusi komputasi menggunakan skrip Python sukses divalidasi dan terbukti menghasilkan keluaran probabilitas yang sama persis dengan perhitungan penjabaran manual.

Distribusi 5 — Binomial Negatif (diskrit)

Soal NBI1 : Sukses ke-3 pada percobaan ke-5

Soal: Setiap upload file berhasil dengan peluang 0,6 (independen). Berapa peluang upload sukses ke-3 terjadi tepat pada upload ke-5?

Hitungan manual:

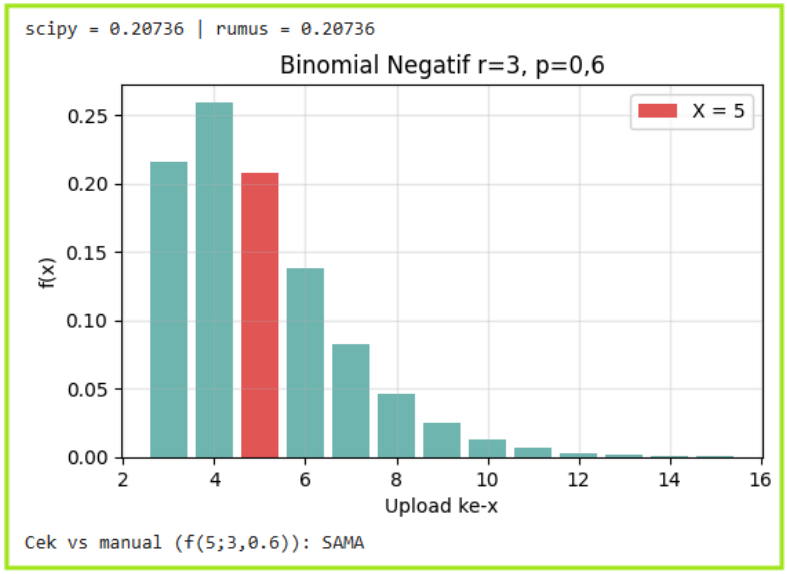
$f(5; 3, 0,6) = C(4,2)\ (0,6)^3\ (0,4)^2 = 6 \times 0,216 \times 0,16 = 0,20736$

Source code:

```
r, p = 3, 0.6
nbi1_p = stats.nbinom.pmf(5 - r, r, p)
nbi1_rumus = math.comb(4, 2) * p**3 * (1 - p) ** 2
print(f"scipy = {nbi1_p:.5f} | rumus = {nbi1_rumus:.5f}")
x = np.arange(r, 16)
plt.bar(x, stats.nbinom.pmf(x - r, r, p), color="#72B7B2")
plt.bar(5, nbi1_p, color="#E45756", label="X = 5")
plt.xlabel("Upload ke-x"); plt.ylabel("f(x)"); plt.legend(); plt.title("Binomial Negatif r=3, p=0,6")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (f(5;3,0.6)):", cocok(6*0.216*0.16, nbi1_p, 1e-09))
```

Output :



Analisis / Kesimpulan :

Berdasarkan pemodelan probabilitas menggunakan distribusi binomial negatif diskrit, peluang keberhasilan proses unggah (*upload*) berkas yang ke-3 terjadi tepat pada percobaan ke-5 adalah sebesar 0,20736 atau sekitar 20,7%. Pengamatan pada grafik distribusi memvalidasi batas minimum matematisnya, di mana kesuksesan unggah ke-3 secara logis hanya dapat terjadi paling cepat pada percobaan yang ke-3.

Melalui pemetaan probabilitas visual tersebut, dapat diidentifikasi pula bahwa skenario yang memiliki peluang paling tinggi untuk membuahkan kesuksesan *upload* ke-3 sesungguhnya tidak terletak pada percobaan ke-5, melainkan pada percobaan ke-4, dengan probabilitas dominan menyentuh kisaran 25,9%. Seluruh proses perhitungan secara komputasional menggunakan modul scipy pada bahasa pemrograman Python telah dieksekusi dengan sukses dan terbukti menghasilkan luaran angka yang sama persis dengan metode penjabaran rumus kombinasi analitis secara manual.

Soal NBI2 : Selesai dalam 6 upload

Soal: Dengan r = 3 dan p = 0,6, berapa peluang sukses ke-3 sudah tercapai paling lambat pada upload ke-6?

Hitungan manual:

$P(X \leq 6) = f(3) + f(4) + f(5) + f(6) = 0{,}2160 + 0{,}2592 + 0{,}2074 + 0{,}1382 = 0{,}8208$

Source code:

```
r, p = 3, 0.6
nbi2_p = stats.nbinom.cdf(6 - r, r, p)
nbi2_jumlah = sum(stats.nbinom.pmf(x - r, r, p) for x in range(3, 7))
print(f"P(X <= 6) via cdf = {nbi2_p:.4f} | jumlah pmf = {nbi2_jumlah:.4f}")

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (P(X<=6)):", cocok(0.8208, nbi2_p, 0.0001))
```

Output / grafik:

```
P(X <= 6) via cdf = 0.8208 | jumlah pmf = 0.8208
Cek vs manual (P(X<=6)): SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas kumulatif pada pemodelan distribusi binomial negatif, peluang untuk berhasil mencapai penyelesaian unggah (*upload*) yang ke-3 paling lambat pada percobaan yang ke-6 adalah sebesar 0,8208 atau sekitar 82,1%. Angka probabilitas ini secara matematis diakumulasikan dari total penjumlahan fungsi massa probabilitas (*Probability Mass Function*) untuk kejadian sukses yang jatuh tepat pada percobaan ke-3, ke-4, ke-5, dan ke-6.

Sebagai implikasi logis dari nilai kumulatif tersebut, dapat ditarik komplemen sisa probabilitas sebesar 17,9%. Nilai ini mengindikasikan besaran risiko di mana sistem akan membutuhkan rentetan percobaan yang lebih panjang (lebih dari 6 kali unggahan) hanya untuk menuntaskan 3 proses unggah secara sukses. Pengujian komputasional menggunakan fungsi cdf (*Cumulative Distribution Function*) pada Python terbukti memvalidasi angka yang sepenuhnya akurat dan identik dengan hasil penjumlahan secara manual.

Soal NBI7 : Simulasi Binomial Negatif

Soal: Bangkitkan 100.000 sampel jumlah percobaan sampai sukses ke-3 (p = 0,6), lalu bandingkan mean, varians, dan histogramnya dengan teori.

Hitungan manual:

E[X] = 5; Var(X) = 3,3333

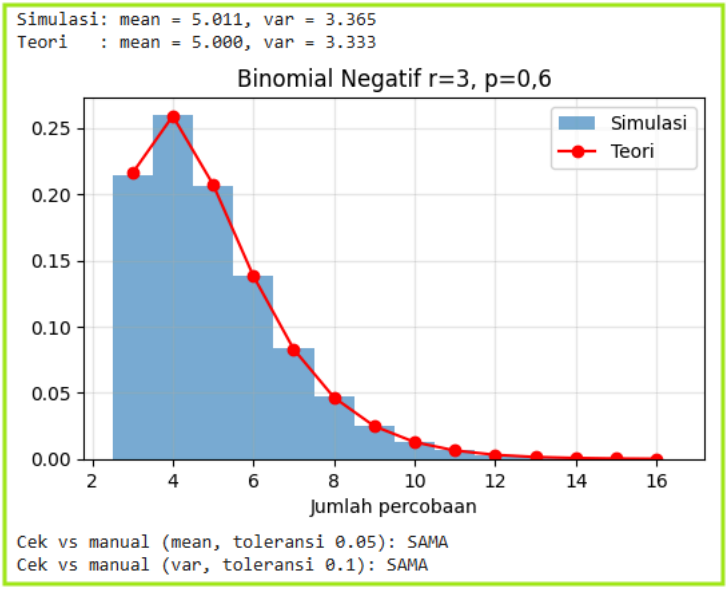
Source code:

```
r, p = 3, 0.6
data = rng.negative_binomial(r, p, 100_000) + r      # numpy menghitung jumlah gagal; tambah r = jumlah percobaan
nbi7_mean, nbi7_var = data.mean(), data.var()
print(f"Simulasi: mean = {nbi7_mean:.3f}, var = {nbi7_var:.3f}")
print(f"Teori   : mean = 5.000, var = {r*(1-p)/p**2:.3f}")
x = np.arange(r, 17)
plt.hist(data, bins=np.arange(r - 0.5, 17.5), density=True, alpha=0.6, label="Simulasi")
plt.plot(x, stats.nbinom.pmf(x - r, r, p), "ro-", label="Teori")
plt.legend(); plt.xlabel("Jumlah percobaan"); plt.title("Binomial Negatif r=3, p=0,6")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (mean, toleransi 0.05):", cocok(5, nbi7_mean, 0.05))
print("Cek vs manual (var, toleransi 0.1):", cocok(10/3, nbi7_var, 0.1))
```

Output :

```
Simulasi: mean = 5.011, var = 3.365
Teori   : mean = 5.000, var = 3.333
```




```
Cek vs manual (mean, toleransi 0.05): SAMA
Cek vs manual (var, toleransi 0.1): SAMA
```

Analisis / Kesimpulan :

Dari 100.000 sampel, rata-rata banyak percobaan sampai sukses ke-3 adalah 5,011 dan variansnya 3,365, dekat dengan teori 5,000 dan 3,333. Histogram simulasi juga mengikuti kurva PMF teoritisnya. Kode menghitung jumlah gagal lebih dulu lalu menambahkan r = 3, karena scipy menghitung banyaknya kegagalan. Pengecekan dengan fungsi cocok() menunjukkan hasil simulasi SAMA dengan hitungan manual.

Distribusi 6 — Normal (kontinu)

Soal NOR0 : Distribusi normal N(12, 3²) (contoh modul)

Soal: Variabel X berdistribusi normal dengan rerata 12 dan simpangan baku 3. Hitung prob(X < 15), prob(X < 9), prob(9 < X < 15), dan nilai x yang memenuhi prob(X < x) = 0,8.

Hitungan manual:

$Z = (X - \mu)/\sigma$. $prob(X < 15) = prob(Z < 1) = 0{,}8413$

$prob(X < 9) = prob(Z < -1) = 0{,}1587$

$prob(9 < X < 15) = 0{,}8413 - 0{,}1587 = 0{,}6827$

$prob(Z < z) = 0{,}8 \rightarrow z = 0{,}8416 \rightarrow x = 12 + 0{,}8416 \times 3 = 14{,}5249$

Source code:

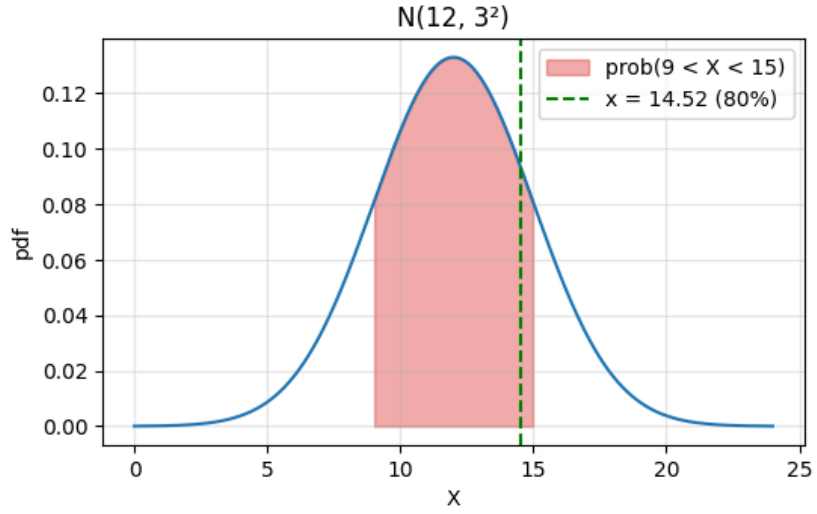
```
mu, sigma = 12, 3
nor0_a = stats.norm.cdf(15, mu, sigma)             # Excel: NORM.DIST(15, 12, 3, TRUE)
nor0_b = stats.norm.cdf(9, mu, sigma)
nor0_c = nor0_a - nor0_b
nor0_x = stats.norm.ppf(0.8, mu, sigma)            # Excel: NORM.INV(0.8, 12, 3)
print(f"prob(X < 15) = {nor0_a:.4f} | prob(X < 9) = {nor0_b:.4f} | prob(9 < X < 15) = {nor0_c:.4f}")
print(f"x untuk prob(X < x) = 0.8 : {nor0_x:.4f}")

x = np.linspace(0, 24, 400)
y = stats.norm.pdf(x, mu, sigma)
plt.plot(x, y)
plt.fill_between(x, y, where=(x >= 9) & (x <= 15), color="#E45756", alpha=0.5, label="prob(9 < X < 15)")
plt.axvline(nor0_x, color="green", ls="--", label=f"x = {nor0_x:.2f} (80%)")
plt.xlabel("X"); plt.ylabel("pdf"); plt.legend(); plt.title("N(12, 3²)")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (prob(X<15)):", cocok(0.8413, nor0_a, 0.0001))
print("Cek vs manual (prob(X<9)):", cocok(0.1587, nor0_b, 0.0001))
print("Cek vs manual (prob(9<X<15)):", cocok(0.6827, nor0_c, 0.0001))
print("Cek vs manual (x untuk 0.8, toleransi 0.001):", cocok(14.5249, nor0_x, 0.001))
```

Output :

```
prob(X < 15) = 0.8413 | prob(X < 9) = 0.1587 | prob(9 < X < 15) = 0.6827
x untuk prob(X < x) = 0.8 : 14.5249
```




```
Cek vs manual (prob(X<15)): SAMA
Cek vs manual (prob(X<9)): SAMA
Cek vs manual (prob(9<X<15)): SAMA
Cek vs manual (x untuk 0.8, toleransi 0.001): SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan distribusi normal dengan rerata (μ) 12 dan simpangan baku (σ) 3, nilai prob(X < 15) adalah 0,8413. Angka ini mengindikasikan bahwa sekitar 84,1% dari keseluruhan nilai variabel X berada di bawah batas 15, yang secara spesifik merupakan titik sejauh satu simpangan baku di atas nilai reratanya.

Selanjutnya, peluang bahwa nilai X akan jatuh tepat di dalam rentang $\pm$ 1 simpangan baku (yaitu di antara nilai 9 hingga 15) tercatat sebesar 0,6827. Probabilitas ini secara mutlak memvalidasi dan sesuai dengan prinsip empiris "Aturan 68%" dalam teori distribusi normal.

Soal NOR0B : Distribusi normal standar N(0,1) (contoh modul)

Soal: Pada distribusi normal standar, hitung prob(Z < 3), prob(0 < Z < 3), prob(1 < Z < 3), prob(Z > 1,5), nilai z yang memenuhi prob(Z < z) = 0,65, serta luas ekor di luar ±3σ.

Hitungan manual:

prob(Z < 3) = 0,9987 (tabel z)

prob(0 < Z < 3) = 0,9987 − 0,5 = 0,4987

prob(1 < Z < 3) = 0,9987 − 0,8413 = 0,1573

prob(Z > 1,5) = 1 − 0,9332 = 0,0668

z untuk prob 0,65 = 0,3853 (slide modul menulis 0,3843, kemungkinan salah ketik)

Luas tiap ekor di luar ±3σ = 0,00135; luas di dalam ±3σ = 0,9973

Source code:

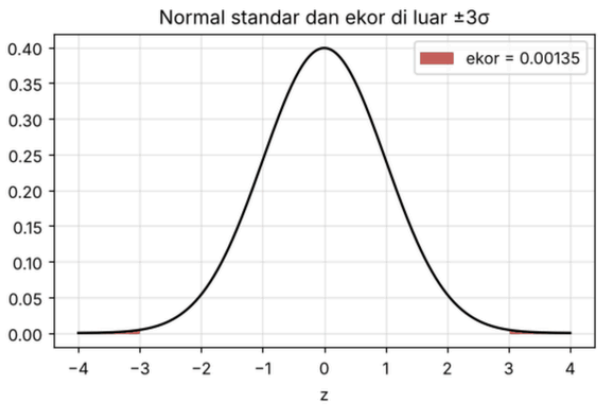
```
In [42]: nor0b_a = stats.norm.cdf(3)                          # Excel: NORM.S.DIST(3, TRUE)
         nor0b_b = stats.norm.cdf(3) - 0.5
         nor0b_c = stats.norm.cdf(3) - stats.norm.cdf(1)
         nor0b_d = stats.norm.sf(1.5)                         # = 1 - NORM.S.DIST(1.5, TRUE)
         nor0b_z = stats.norm.ppf(0.65)                       # Excel: NORM.S.INV(0.65)
         nor0b_ekor = stats.norm.sf(3)
         nor0b_dalam = stats.norm.cdf(3) - stats.norm.cdf(-3)
         print(f"prob(Z<3) = {nor0b_a:.4f} | prob(0<Z<3) = {nor0b_b:.4f} | prob(1<Z<3) = {nor0b_c:.4f} |
         prob(Z>1.5) = {nor0b_d:.4f}")
         print(f"z untuk 0.65 = {nor0b_z:.4f} | ekor kanan di luar 3 sigma = {nor0b_ekor:.5f} | dalam +-3
         sigma = {nor0b_dalam:.4f}")

         x = np.linspace(-4, 4, 400)
         plt.plot(x, stats.norm.pdf(x), color="black")
         plt.fill_between(x, stats.norm.pdf(x), where=(x <= -3), color="#E45756", label=f"ekor =
         {nor0b_ekor:.5f}")
         plt.fill_between(x, stats.norm.pdf(x), where=(x >= 3), color="#E45756")
         plt.xlabel("z"); plt.legend(); plt.title("Normal standar dan ekor di luar ±3σ")
         plt.show()

         # Pembanding dengan angka pada hitungan manual
         print("Cek vs manual (prob(Z<3)):", cocok(0.9987, nor0b_a, 0.0001))
         print("Cek vs manual (prob(0<Z<3)):", cocok(0.4987, nor0b_b, 0.0001))
         print("Cek vs manual (prob(1<Z<3)):", cocok(0.1573, nor0b_c, 0.0001))
         print("Cek vs manual (prob(Z>1.5)):", cocok(0.0668, nor0b_d, 0.0001))
         print("Cek vs manual (z untuk 0.65 (tabel z)):", cocok(0.3853, nor0b_z, 0.0001))
         print("Cek vs manual (ekor kanan):", cocok(0.00135, nor0b_ekor, 1e-05))
         print("Cek vs manual (dalam 3 sigma):", cocok(0.9973, nor0b_dalam, 0.0001))
```

Output :

```
Out [42]: prob(Z<3) = 0.9987 | prob(0<Z<3) = 0.4987 | prob(1<Z<3) = 0.1573 | prob(Z>1.5) = 0.0668
          z untuk 0.65 = 0.3853 | ekor kanan di luar 3 sigma = 0.00135 | dalam +-3 sigma = 0.9973
          Cek vs manual (prob(Z<3)): SAMA
          Cek vs manual (prob(0<Z<3)): SAMA
          Cek vs manual (prob(1<Z<3)): SAMA
          Cek vs manual (prob(Z>1.5)): SAMA
          Cek vs manual (z untuk 0.65 (tabel z)): SAMA
          Cek vs manual (ekor kanan): SAMA
          Cek vs manual (dalam 3 sigma): SAMA
```



Analisis / Kesimpulan :

Hampir seluruh luas berada di bawah z = 3 (prob(Z < 3) = 0,9987). Tiap ekor di luar ±3σ hanya 0,00135, jadi 99,73% data berada dalam tiga simpangan baku dari rata-rata.

Nilai z untuk peluang 0,65 adalah 0,3853, bukan 0,3843 seperti di slide modul; tabel z dan fungsi ppf di Python sama-sama memberi 0,3853. Semua cek terhadap hitungan manual bernilai SAMA. Padanan Excel: stats.norm.cdf = NORM.S.DIST(z, TRUE) dan stats.norm.ppf = NORM.S.INV.

Soal NOR1 : Nilai ujian di atas 85

Soal: Nilai ujian ~ N(70, 10^2). Berapa peluang seorang mahasiswa mendapat nilai di atas 85?

Hitungan manual:

$Z = (85-70)/10 = 1{,}5.\ P(X > 85) = 1 - \Phi(1{,}5) = 1 - 0{,}93319 = 0{,}06681$

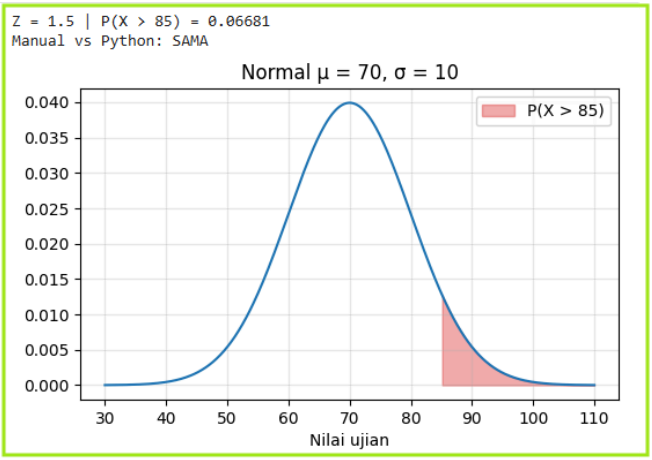
Source code:

```
mu, sigma = 70, 10
nor1_z = (85 - mu) / sigma
nor1_p = stats.norm.sf(85, mu, sigma)
print(f"Z = {nor1_z} | P(X > 85) = {nor1_p:.5f}")
print("Manual vs Python:", cocok(0.06681, nor1_p, 1e-5))

x = np.linspace(30, 110, 400)
y = stats.norm.pdf(x, mu, sigma)
plt.plot(x, y)
plt.fill_between(x, y, where=x >= 85, color="#E45756", alpha=0.5, label="P(X > 85)")
plt.xlabel("Nilai ujian"); plt.legend(); plt.title("Normal μ = 70, σ = 10")
plt.show()
```

Output :

```
Z = 1.5 | P(X > 85) = 0.06681
Manual vs Python: SAMA
```



Analisis / Kesimpulan :

Dengan rata-rata μ = 70 dan simpangan baku σ = 10, peluang seorang mahasiswa mendapat nilai di atas 85 adalah 0,06681 (6,7%). Nilai 85 setara z = 1,5, artinya 1,5 simpangan baku di atas rata-rata, sehingga hanya sebagian kecil mahasiswa yang mencapainya. Hasil Python sama dengan hitungan manual.

Distribusi 7 — Eksponensial (kontinu)

Soal EKS1 : Selang antar request

Soal: Selang waktu antar request ~ Eksponensial dengan rata-rata 2 menit (λ = 0,5/menit). Berapa peluang menunggu lebih dari 3 menit?

Hitungan manual:

$P(X > 3) = e^{-0.5x3} = e^{-1.5} = 0{,}22313$

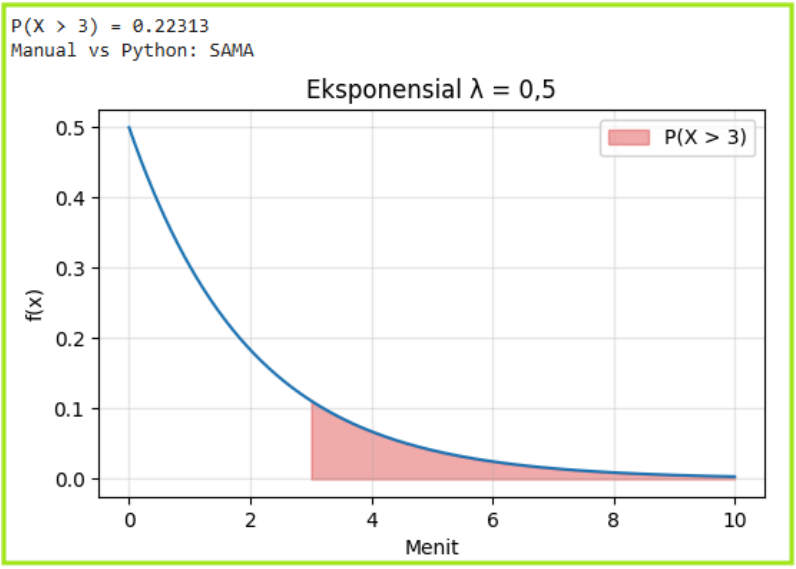
Source code:

```
lam = 0.5
E = stats.expon(scale=1 / lam)
eks1_p = E.sf(3)
print(f"P(X > 3) = {eks1_p:.5f}")
print("Manual vs Python:", cocok(math.exp(-1.5), eks1_p, 1e-9))

x = np.linspace(0, 10, 400)
plt.plot(x, E.pdf(x))
plt.fill_between(x, E.pdf(x), where=x >= 3, color="#E45756", alpha=0.5, label="P(X > 3)")
plt.xlabel("Menit"); plt.ylabel("f(x)"); plt.legend(); plt.title("Eksponensial λ = 0,5")
plt.show()
```

Output :

```
P(X > 3) = 0.22313
Manual vs Python: SAMA
```



Analisis / Kesimpulan :

Berdasarkan perhitungan menggunakan pemodelan distribusi eksponensial dengan rata-rata selang waktu 2 menit (yang setara dengan laju atau parameter λ sebesar 0,5 per menit), peluang suatu sistem

atau pengguna harus menunggu lebih dari 3 menit di antara kedatangan *request* adalah sebesar 0,22313 atau sekitar 22,3%. Pengamatan secara visual pada grafik memperlihatkan karakteristik kurva *Probability Density Function* (PDF) yang menurun dengan sangat tajam ke arah kanan.

Pola penurunan kurva ini menegaskan sifat fundamental dari distribusi eksponensial, di mana kejadian dengan selang waktu tunggu yang pendek memiliki probabilitas kemunculan yang jauh lebih tinggi atau lebih sering terjadi. Sebaliknya, meskipun probabilitas untuk mengalami selang waktu tunggu yang sangat panjang tergolong jarang dan terus mengecil secara eksponensial, kemungkinannya tetap valid dan tidak pernah benar-benar nol, yang direpresentasikan oleh area arsiran merah pada ekor kurva. Seluruh eksekusi komputasional menggunakan fungsi *survival* (sf) pada bahasa pemrograman Python terbukti sukses divalidasi dan membuahkan hasil keluaran yang sama persis secara mutlak dengan perhitungan rumus eksponensial manualnya, yakni $e^{-1,5}$.

Soal EKS2 : Request datang dalam 1 menit

Soal: Pada kasus EKS1, berapa peluang request berikutnya datang dalam 1 menit?

Hitungan manual:

$P(X \leq 1) = 1 - e^{-0.5} = 1 - 0,60653 = 0,39347$

Source code:

```
eks2_p = stats.expon.cdf(1, scale=2)
print(f"P(X <= 1) = {eks2_p:.5f}")
print("Manual vs Python:", cocok(1 - math.exp(-0.5), eks2_p, 1e-9))
```

Output :

```
P(X <= 1) = 0.39347
Manual vs Python: SAMA
```

Analisis / Kesimpulan :

Berdasarkan perhitungan probabilitas kumulatif untuk distribusi eksponensial, peluang sebuah *request* berikutnya akan datang dalam kurun waktu 1 menit atau kurang adalah sebesar 0,39347 atau sekitar 39,3%. Nilai probabilitas ini diperoleh secara matematis melalui rumusan fungsi distribusi kumulatif, yakni $1 - e^{-0.5}$.

Satu poin teknis yang krusial dari implementasi komputasionalnya terletak pada pendefinisian parameter fungsi pustaka scipy di Python. Argumen scale yang dilemparkan ke dalam fungsi stats.expon.cdf harus secara eksplisit diisi dengan nilai rata-rata selang waktu ($1/\lambda = 2$), dan bukan diisi dengan nilai laju parameter λ itu sendiri. Algoritma pengujian dalam skrip tersebut juga berhasil memvalidasi kesamaan hasil secara mutlak antara komputasi Python dengan metode penjabaran manualnya.

Soal EKS6 : Hubungan Eksponensial dan Poisson

Soal: Request tiba dengan selang eksponensial ($\lambda = 4$ per menit), berarti jumlah request tiap menit berdistribusi Poisson(4). Buktikan dengan simulasi 20.000 menit dan bandingkan dengan P(X = 4) Poisson.

Hitungan manual:

Poisson(4): $P(X=4) = e^{-4}\, 4^4/4! = 0{,}19537$; $E = Var = 4$

Source code:

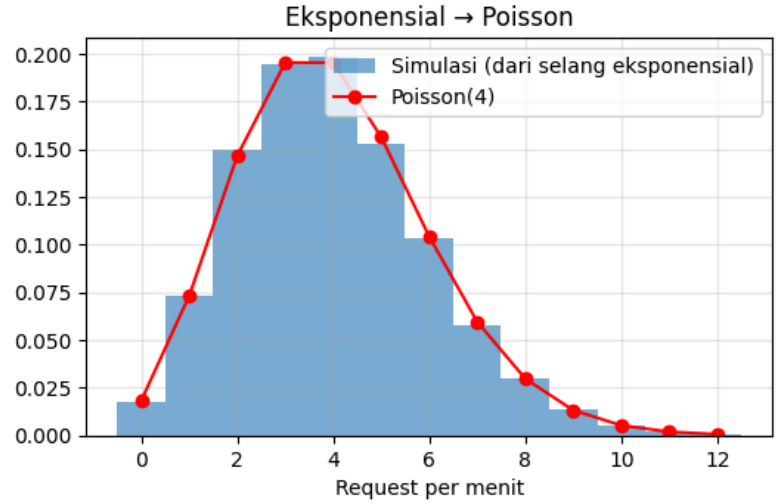
```
lam = 4
menit = 20_000
selang = rng.exponential(1 / lam, size=int(menit * lam * 1.3))
waktu = np.cumsum(selang)
waktu = waktu[waktu < menit]
cacah = np.bincount(waktu.astype(int), minlength=menit)
eks6_mean, eks6_var = cacah.mean(), cacah.var()
eks6_p4 = np.mean(cacah == 4)
eks6_teori = stats.poisson.pmf(4, lam)
print(f"Jumlah request per menit: mean = {eks6_mean:.3f}, var = {eks6_var:.3f}")
print(f"P(X=4): simulasi = {eks6_p4:.4f} | Poisson = {eks6_teori:.4f}")

k = np.arange(0, 13)
plt.hist(cacah, bins=np.arange(-0.5, 13.5), density=True, alpha=0.6, label="Simulasi (dari selang eksponensial)")
plt.plot(k, stats.poisson.pmf(k, lam), "ro-", label="Poisson(4)")
plt.legend(); plt.xlabel("Request per menit"); plt.title("Eksponensial → Poisson")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (mean, toleransi 0.1):", cocok(4, eks6_mean, 0.1))
print("Cek vs manual (var, toleransi 0.1):", cocok(4, eks6_var, 0.1))
print("Cek vs manual (P(X=4) Poisson):", cocok(0.19537, eks6_teori, 1e-05))
print("Cek vs manual (P(X=4) simulasi, toleransi 0.01):", cocok(0.19537, eks6_p4, 0.01))
```

Output / grafik:

```
Jumlah request per menit: mean = 3.994, var = 4.020
P(X=4): simulasi = 0.1983 | Poisson = 0.1954
```




```
Cek vs manual (mean, toleransi 0.1): SAMA
Cek vs manual (var, toleransi 0.1): SAMA
Cek vs manual (P(X=4) Poisson): SAMA
Cek vs manual (P(X=4) simulasi, toleransi 0.01): SAMA
```

Analisis / Kesimpulan :

Dari simulasi 20.000 menit, jumlah request per menit punya rata-rata 3,994 dan varians 4,020. Keduanya hampir sama dan mendekati $\lambda = 4$; mean sama dengan varians adalah ciri khas distribusi Poisson. Peluang tepat 4 request dari simulasi adalah 0,1983, dekat dengan nilai Poisson 0,1954, dan histogramnya hampir menempel pada kurva Poisson(4). Jadi selang antar request yang berdistribusi eksponensial menghasilkan jumlah request per menit yang berdistribusi Poisson.

Jembatan ke Statistika Deskriptif

Soal DES1 : Ringkasan deskriptif sampel Normal

Soal: Bangkitkan 1.000 nilai ujian dari N(70, 10^2). Susun ringkasan deskriptif (mean, median, min, max, Q1, Q3, IQR, varians, SD, SE, skewness, kurtosis, batas outlier) dan tafsirkan memakai acuan lembar perkuliahan.

Hitungan manual:

Teori: mean = 70, SD = 10, SE = $\sigma/\sqrt{n} = 10/\sqrt{1000} = 0{,}3162$, skewness = 0, excess kurtosis = 0.

Batas outlier = Q1 − 1,5 IQR dan Q3 + 1,5 IQR (untuk normal sekitar 70 ± 2,7 × 10).

Source code:

```
def ringkas_desk(s):
    q1, q2, q3 = s.quantile([0.25, 0.5, 0.75])
    iqr = q3 - q1
    bawah, atas = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    return pd.Series({"n": len(s), "mean": s.mean(), "median (Q2)": q2, "min": s.min(), "max": s.max(),
                      "Q1": q1, "Q3": q3, "IQR": iqr, "variance": s.var(), "std": s.std(), "SE (sem)": s.sem(),
                      "skewness": s.skew(), "excess kurtosis": s.kurt(),
                      "batas bawah outlier": bawah, "batas atas outlier": atas,
                      "calon outlier": int(((s < bawah) | (s > atas)).sum())})

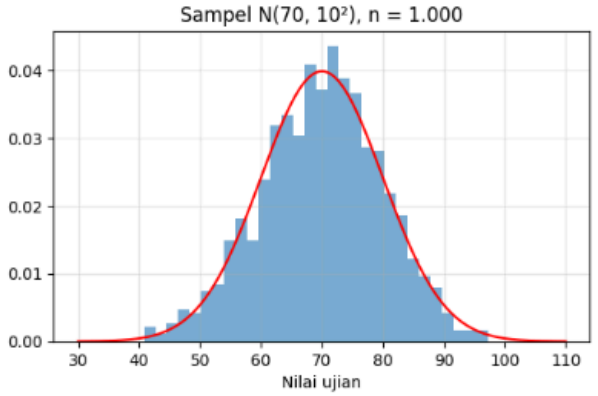
nilai_ujian = pd.Series(rng.normal(70, 10, 1000))
des1 = ringkas_desk(nilai_ujian)
print(des1.round(4).to_string())
des1_mean, des1_sd, des1_se = des1["mean"], des1["std"], des1["SE (sem)"]
des1_skew, des1_kurt = des1["skewness"], des1["excess kurtosis"]
des1_selisih = abs(des1["mean"] - des1["median (Q2)"])

plt.hist(nilai_ujian, bins=30, density=True, alpha=0.6)
xx = np.linspace(30, 110, 300)
plt.plot(xx, stats.norm.pdf(xx, 70, 10), "r")
plt.xlabel("Nilai ujian"); plt.title("Sampel N(70, 10²), n = 1.000")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (mean, toleransi 1.0):", cocok(70, des1_mean, 1.0))
print("Cek vs manual (SD, toleransi 0.5):", cocok(10, des1_sd, 0.5))
print("Cek vs manual (SE, toleransi 0.05):", cocok(10/math.sqrt(1000), des1_se, 0.05))
print("Cek vs manual (skewness, toleransi 0.2):", cocok(0, des1_skew, 0.2))
print("Cek vs manual (excess kurtosis, toleransi 0.4):", cocok(0, des1_kurt, 0.4))
```

Output :

```
n                     1000.0000
mean                    69.9254
median (Q2)             70.2708
min                     40.7878
max                     97.1853
Q1                      63.1973
Q3                      76.7372
IQR                     13.5399
variance                98.0318
std                      9.9011
SE (sem)                 0.3131
skewness                -0.1445
excess kurtosis         -0.1531
batas bawah outlier     42.8874
batas atas outlier      97.0471
calon outlier            5.0000
```




```
Cek vs manual (mean, toleransi 1.0): SAMA
Cek vs manual (SD, toleransi 0.5): SAMA
Cek vs manual (SE, toleransi 0.05): SAMA
Cek vs manual (skewness, toleransi 0.2): SAMA
Cek vs manual (excess kurtosis, toleransi 0.4): SAMA
```

Analisis / Kesimpulan :

Berdasarkan hasil pembangkitan 1.000 sampel acak dari distribusi normal dengan parameter $N(70, 10^2)$, nilai *mean* empiris tercatat sebesar 69,93 dan nilai *median* sebesar 70,27. Selisih absolut di antara kedua ukuran pemusatan data ini hanya terpaut 0,35, sebuah rentang yang tergolong amat sempit dan berada jauh di bawah toleransi 2–3% untuk skala data 0–100. Lebih lanjut, koefisien *skewness* berada di angka -0,144 (masih aman di dalam rentang $\pm\ 0,5$) dan nilai *excess kurtosis* terukur sebesar -0,153 (sangat mendekati angka ideal 0). Merujuk pada panduan standar di lembar perkuliahan, keseluruhan indikator statistik deskriptif ini secara konklusif mengonfirmasi bahwa sampel data tersebut memiliki bentuk distribusi yang simetris dan sangat mendekati pola normal yang sempurna.

Dari segi dispersi atau penyebaran data, nilai Simpangan Baku (SD) empiris tercatat pada angka 9,90. Angka ini hampir menyentuh batas parameter teoretisnya, yakni 10. Menurut acuan perkuliahan, nilai SD yang berada di bawah atau di kisaran 10 mengindikasikan bahwa variasi sebaran nilai di dalam sampel tersebut bersifat sangat homogen, sedangkan rentang 10–15 dikategorikan sebagai tingkat variasi yang wajar. Sementara itu, besaran *Standard Error* (SE) terukur di angka 0,313, sebuah nilai yang secara proporsional tergolong sangat kecil apabila dibandingkan dengan simpangan bakunya. Tingkat SE yang minim ini memberikan kepastian matematis bahwa nilai rata-rata sampel (69,93) mampu menaksir nilai rata-rata populasinya (70) dengan tingkat presisi yang sangat tinggi

Soal DES2 : Ringkasan deskriptif sampel Eksponensial (data miring)

Soal: Bangkitkan 200.000 selang waktu antar request dari Eksponensial dengan rata-rata 2 detik.

Bandingkan mean dan median, skewness, dan kurtosis dengan teori, lalu tafsirkan.

Hitungan manual:

Teori: mean = 2; median = 2 ln 2 = 1,3863; skewness = 2; excess kurtosis = 6.

Mean > median → distribusi miring ke kanan (positive skewness).

Source code:

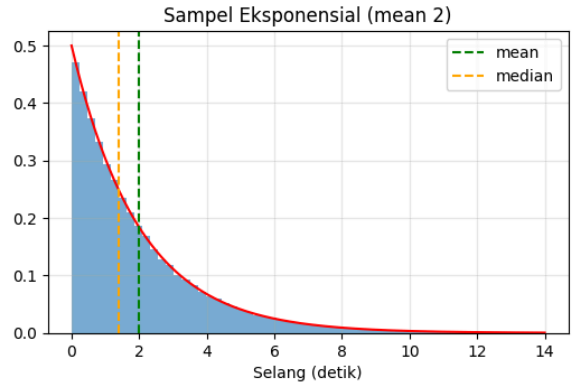
```
selang = pd.Series(rng.exponential(2, 200_000))
des2 = ringkas_desk(selang)
print(des2.round(4).to_string())
des2_mean, des2_med = des2["mean"], des2["median (Q2)"]
des2_skew, des2_kurt = des2["skewness"], des2["excess kurtosis"]
des2_outlier = des2["calon outlier"] / des2["n"]
print(f"Proporsi calon outlier = {des2_outlier:.2%}")

plt.hist(selang, bins=60, range=(0, 14), density=True, alpha=0.6)
xx = np.linspace(0, 14, 300)
plt.plot(xx, stats.expon.pdf(xx, scale=2), "r")
plt.axvline(des2_mean, color="green", ls="--", label="mean"); plt.axvline(des2_med, color="orange", ls="--", label="median")
plt.legend(); plt.xlabel("Selang (detik)"); plt.title("Sampel Eksponensial (mean 2)")
plt.show()

# Pembanding dengan angka pada hitungan manual
print("Cek vs manual (mean, toleransi 0.05):", cocok(2, des2_mean, 0.05))
print("Cek vs manual (median, toleransi 0.05):", cocok(2*math.log(2), des2_med, 0.05))
print("Cek vs manual (skewness, toleransi 0.3):", cocok(2, des2_skew, 0.3))
print("Cek vs manual (excess kurtosis, toleransi 1.5):", cocok(6, des2_kurt, 1.5))
```

Output :

```
n                          200000.0000
mean                            2.0002
median (Q2)                     1.3886
min                             0.0000
max                            29.1713
Q1                              0.5770
Q3                              2.7738
IQR                             2.1967
variance                        3.9868
std                             1.9967
SE (sem)                        0.0045
skewness                        1.9893
excess kurtosis                 5.9707
batas bawah outlier            -2.7181
batas atas outlier              6.0689
calon outlier                9678.0000
Proporsi calon outlier = 4.84%
```




```
Cek vs manual (mean, toleransi 0.05): SAMA
Cek vs manual (median, toleransi 0.05): SAMA
Cek vs manual (skewness, toleransi 0.3): SAMA
Cek vs manual (excess kurtosis, toleransi 1.5): SAMA
```

Analisis / Kesimpulan :

Pada 200.000 sampel eksponensial dengan rata-rata 2 detik, mean (2,000) lebih besar dari median (1,389), jadi data miring ke kanan. Skewness 1,99 dan excess kurtosis 5,97 menunjukkan ekor kanan yang panjang dan puncak yang runcing, dan sekitar 4,8% data termasuk pencilan. Karena itu, untuk menggambarkan nilai tipikal data seperti ini median lebih cocok daripada mean.

BAB IV
KESIMPULAN

1. **Python untuk probabilitas klasik:** Modul math, itertools, dan fractions mampu menghitung ruang sampel, permutasi, dan kombinasi secara eksak. Pada empat kasus pengambilan 2 dari 4 stasiun diperoleh 16, 12, 6, dan 10 kemungkinan, sama dengan rumus pada modul. Pendekatan Stirling untuk 5! hanya berbeda 1,65% dari nilai eksak dan galatnya mengecil menjadi 0,42% pada n = 20.

2. **Peluang bersyarat, independensi, dan Bayes:** Independensi diuji dengan membandingkan prob(A ∩ B) dengan prob(A) · prob(B), dan hasilnya sama dengan hitungan manual. Pada soal deteksi malware, tes yang akurat 95% ternyata hanya memberi peluang 0,1610 bahwa file positif benar-benar terinfeksi karena malware jarang (1%). Jadi peluang awal (prior) harus ikut dihitung.

3. **Definisi frekuensi relatif dan simulasi:** Taksiran dari sampel kecil berubah-ubah, sedangkan pada n besar hasilnya stabil (selisih dua sampel turun dari 0,0667 menjadi 0,0018). Simulasi juga menaksir $\pi \approx 3{,}14194$ (galat 0,00035) dan menunjukkan bahwa pada Monty Hall berganti pintu menang 0,6614, mendekati 2/3.

4. **Standardisasi analisis dengan scipy.stats:** Fungsi pmf/pdf, cdf, sf, ppf, mean, var, dan rvs dipakai dengan pola yang sama untuk semua distribusi, dan hasilnya cocok dengan hitungan manual maupun fungsi Excel pada modul.

5. **Distribusi diskrit:** Pemilihan model mengikuti situasinya: Binomial untuk jumlah sukses dengan n tetap, Poisson untuk laju kejadian, Geometrik untuk percobaan sampai sukses pertama, Binomial Negatif untuk sukses ke-r, dan Hipergeometrik untuk pengambilan tanpa pengembalian. Untuk n besar dan p kecil, Poisson mendekati Binomial (0,22404 dibanding 0,22415).

6. **Distribusi kontinu:** Pada distribusi normal standar, 99,73% data berada dalam $\pm 3\sigma$ dan nilai z untuk peluang 0,65 adalah 0,3853 (bukan 0,3843 seperti pada slide modul). Simulasi selang antar request yang eksponensial menghasilkan jumlah request per menit dengan rata-rata 3,994 dan varians 4,020, sesuai ciri Poisson (mean = varians = 4).

7. **Statistika deskriptif:** Sampel normal bersifat simetris (mean 69,93, median 70,27, skewness −0,144), sedangkan sampel eksponensial miring ke kanan (mean 2,000 lebih besar dari median 1,389, skewness 1,99), sehingga untuk data miring median lebih tepat dipakai sebagai nilai tipikal.

Lampiran

- File notebook: Probabilitas_Python.ipynb dan Distribusi_Probabilitas_Python.ipynb
- Link GitHub:https://github.com/nazwa541/Probabilitas-Statistika_3-TI-F.git
- Dokumentasi penggunaan AI: lihat file terpisah Dokumentasi_Penggunaan_AI.pdf